

From Platonic Solids to Regular 4-Polytopes: A Topological and Coxeter-Theoretic Classification with Lean 4 Certificate generated by LLM and registered in Palomar

Lars Warren Ericson¹

¹ORCID: 0000-0001-8299-9361

¹Catskills Research Company

¹lars.ericson@catskillsresearch.com

September 13, 2026

Github: <https://github.com/catskillsresearch/platonic-solids>

Abstract

We present an expository and formally verified classification of the regular convex polyhedra in $\mathbb{R}^3$ (the Platonic solids) and the regular convex polychora in $\mathbb{R}^4$ formalized in **Lean 4**. In dimension 3, we formalize the deduction from the topological Euler characteristic of the 2-sphere ($\chi(S^2) = 2$) and incidence combinatorics to the Diophantine inequality $(p-2)(q-2) < 4$, proving the existence of exactly 5 solutions. A key formalization result is the complete elimination of the classical non-degeneracy axiom: the topological and regularity conditions alone algebraically force $E > 0$ without geometric postulation. In dimension 4, where the Euler–Poincaré characteristic vanishes ($\chi(S^3) = 0$) and topology yields no upper bound on flags, we discard all heuristic combinatorial filters and classify the polychora via **Coxeter reflection groups and Gram matrices**. By Sylvester’s criterion, spherical 4-polytopes correspond bijectively to positive-definite Schläfli matrices. We compute the leading principal minors directly and verify that out of the 11 candidate triples formed by Platonic cells and vertex figures, exactly 6 yield positive determinants, 1 yields the parabolic cubic honeycomb ($\det = 0$), and 4 yield hyperbolic honeycombs ($\det < 0$). The accompanying Lean 4 formalization is self-contained, contains **zero axioms, zero admits, and zero sorrys**, deriving all results from first principles. Companion artifact: `PlatonicSolids.lean` (standalone Lake package in this directory).

Contents

1	1. Introduction: The Dimensional Hierarchy	3
2	2. Dimension 3: Topology, Incidence, and the Non-Degeneracy Insight	3
2.1	2.1. Topological Derivation of Euler’s Formula	3
2.2	2.2. Incidence Combinatorics	4
2.3	2.3. The Diophantine Reduction	5
2.4	2.4. Formalization Insight: Elimination of the Non-Degeneracy Axiom	5

3	3. Dimension 4: Why Topology Fails and Coxeter Takes Over	6
3.1	3.1. The Topological Barrier: $\chi(S^3) = 0$	6
3.2	3.2. Coxeter Systems and the Schläfli Gram Matrix	6
3.3	3.3. Sylvester's Criterion and Principal Minors	6
3.4	3.4. The Critical Dimension-4 Minor: D_4	7
4	4. Evaluation of the 11 Candidate Triples	7
4.1	Group I: Spherical Polychora ($\Delta > 0$)	8
4.2	Group II: Euclidean Honeycomb ($\Delta = 0$)	8
4.3	Group III: Hyperbolic Honeycombs ($\Delta < 0$)	8
5	5. Complete Lean 4 Formal Verification	9
6	6. Pedagogical and Formalization Significance	10
7	7. Acknowledgements	11
7.1	Declaration of Generative AI and AI-Assisted Technologies	11
8	8. References	11
A	A. Complete Lean source	11
A.1	PlatonicSolids.lean	12
A.2	PlatonicSolids/RadialProjection.lean	12
A.3	PlatonicSolids/EulerPoincare.lean	13
A.4	PlatonicSolids/SingularHomology.lean	15
A.5	PlatonicSolids/RelativeHomology.lean	17
A.6	PlatonicSolids/MayerVietoris.lean	20
A.7	PlatonicSolids/Sphere2Homology.lean	25
A.8	PlatonicSolids/Sphere3Homology.lean	27
A.9	PlatonicSolids/EulerBetti.lean	30
A.10	PlatonicSolids/PlatonicPair.lean	30
A.11	PlatonicSolids/Reciprocal.lean	31
A.12	PlatonicSolids/Incidence.lean	32
A.13	PlatonicSolids/Angles.lean	33
A.14	PlatonicSolids/GramMatrix.lean	34
A.15	PlatonicSolids/LeadingMinors.lean	34
A.16	PlatonicSolids/TrailingMinor.lean	35
A.17	PlatonicSolids/D3IffPlatonic.lean	35
A.18	PlatonicSolids/QuadForm.lean	36
A.19	PlatonicSolids/CompletingSquare.lean	37
A.20	PlatonicSolids/DetFormula.lean	38
A.21	PlatonicSolids/Sylvester.lean	39
A.22	PlatonicSolids/TrigValues.lean	40
A.23	PlatonicSolids/CompatibleTriples.lean	42
A.24	PlatonicSolids/SignTable.lean	43
A.25	PlatonicSolids/Regular4.lean	45
A.26	PlatonicSolids/Classification.lean	45
A.27	Challenge.lean	47
A.28	Solution.lean	49

1 1. Introduction: The Dimensional Hierarchy

In Euclidean geometry, a regular polytope is a figure whose symmetry group acts transitively on its flags (incident chains of vertices, edges, faces, and higher-dimensional cells).

- In **dimension 2**, there exist infinitely many regular polygons $\{p\}$ for each $p \geq 3$.
- In **dimension 3**, Euclid's *Elements* (Book XIII) establishes that there exist exactly **5 Platonic solids**.
- In **dimension 4**, Ludwig Schläfli proved in 1852 that there exist exactly **6 regular convex 4-polytopes** (polychora).
- In **dimension $n \geq 5$** , the classification collapses permanently to **3 families**: the regular simplex, the hypercube (measure polytope), and the cross-polytope.

This distribution raises foundational mathematical and pedagogical questions: **Why does dimension 3 permit 5 shapes, dimension 4 permit 6 (admitting the exceptional self-dual 24-cell), and dimension $n \geq 5$ permit only 3? Why does topology suffice to bound the classification in 3D, while failing completely in 4D?**

This paper addresses these questions through an analytical exposition coupled with a fully verified formal certificate in Lean 4.

2 2. Dimension 3: Topology, Incidence, and the Non-Degeneracy Insight

2.1 2.1. Topological Derivation of Euler's Formula

The Lean gists are `PlatonicSolids/RadialProjection.lean` (frontier of a convex body $\simeq S^{n-1}$), `PlatonicSolids/EulerPoincare.lean` (algebraic Euler–Poincaré: $\chi(C) = \chi(H_*)$ by rank-nullity), `PlatonicSolids/SingularHomology.lean` (homotopy invariance, H_0 of path-connected spheres, and $H_*(S^0)$), `PlatonicSolids/RelativeHomology.lean` and `PlatonicSolids/MayerVietoris.lean` (pair and triad exact sequences of singular chains), `PlatonicSolids/Sphere2Homology.lean` (cellular $H_*(S^2; \mathbb{Q})$ of the tetrahedron surface), and `PlatonicSolids/EulerBetti.lean` (the numbers $\chi(S^2) = 2$ and $\chi(S^3) = 0$).

Let $P \subset \mathbb{R}^3$ be a convex polyhedron containing the origin in its interior. Radial projection $\pi : \partial P \rightarrow S^2$ onto the circumscribed 2-sphere is a homeomorphism (`frontier_convex_homeo_sphere`). This projection induces a finite CW-complex structure on S^2 with V 0-cells (vertices), E 1-cells (edges), and F 2-cells (faces).

Mathlib already has the singular homology functor and its homotopy invariance. We package the missing transport lemmas: a homotopy equivalence (or homeomorphism) of spaces induces an isomorphism of singular homology (`singularHomologyIso_of_homotopyEquiv`), a contractible space has vanishing positive homology (`isZero_singularHomology_of_contractible`), and a closed disk is contractible. For the metric sphere itself, path-connectedness in dimension $n \geq 1$ gives

$$H_0(S^n; R) \cong R$$

(`singularHomology0_sphere`). The 0-sphere is the two-point set $\{\pm 1\}$, which is totally disconnected, so $H_0(S^0; R) \cong R \oplus R$ and $H_n(S^0; R) = 0$ for $n > 0$.

A subspace inclusion $A \hookrightarrow X$ is a monomorphism in `TopCat`. The singular chain-complex functor preserves monomorphisms, so

$$0 \rightarrow C_*(A; R) \rightarrow C_*(X; R) \rightarrow C_*(X, A; R) \rightarrow 0$$

is short exact (`pair_shortExact`). Snake lemma supplies the long exact sequence of the pair: connecting $\delta : H_{n+1}(X, A) \rightarrow H_n(A)$ (`pair $\delta$`) and exactness at $H_n(A)$, $H_n(X)$, and $H_n(X, A)$ (`pair_exact_at_A`, `pair_exact_at_X`, `pair_exact_at_rel`).

For a commuting square of spaces $W \rightarrow U, V \rightarrow X$, the same argument on $C_*(W) \rightarrow C_*(U) \oplus C_*(V)$ yields the triad sequence (`triad_shortExact`, `triad $\delta$`). The square induces a comparison from the cover chains to $C_*(X)$; the square is *excisive* when that comparison is a quasi-isomorphism (`IsExcisive`). After excision, the connecting map is Mayer–Vietoris for X (`mayerVietoris $\delta$`).

The stereographic cover of S^n is the standard pair of charts $U = S^n \setminus \{N\}$, $V = S^n \setminus \{S\}$. Each is homeomorphic to a hyperplane (`homeoPuncturedNorth`, `homeoPuncturedSouth`), hence contractible, so $H_k(U) = H_k(V) = 0$ for $k > 0$. They cover S^n (`punctured_cover`) and meet in $S^n \setminus \{N, S\}$. If this pair is excisive, Mayer–Vietoris gives the inductive connecting map $H_{n+1}(S^{n+1}) \rightarrow H_n(U \cap V)$ (`singularHomology_sphere_mayerVietoris $\delta$`). The remaining obstruction is exactly that quasi-isomorphism: barycentric subdivision / small simplices, which Mathlib does not have. The classification therefore still uses the cellular calculation below.

The standard simplicial 2-sphere is the surface of a tetrahedron: 4 vertices, 6 edges, 4 triangles. Oriented incidence matrices $d_1 : \mathbb{Q}^6 \rightarrow \mathbb{Q}^4$ and $d_2 : \mathbb{Q}^4 \rightarrow \mathbb{Q}^6$ satisfy $d_1 d_2 = 0$. Each has a 3×3 identity minor, so both ranks are at least 3; the product-zero rank inequality on a 6-dimensional middle term forces both ranks equal to 3. Hence

$$b_0 = 4 - 3 = 1, \quad b_1 = 3 - 3 = 0, \quad b_2 = 4 - 3 = 1$$

(`homology_sphere2`). The Euler characteristic is the alternating sum of Betti numbers:

$$\chi(S^2) = b_0 - b_1 + b_2 = 1 - 0 + 1 = 2$$

Algebraic Euler–Poincaré (`euler_poincare`) is rank-nullity on any 2-complex: cell ranks equal Betti ranks. Specializing to these numbers yields Euler’s formula

$$V - E + F = 2$$

on the tetrahedron (`euler_tetrahedron`) and, for an arbitrary 2-complex with the same Betti numbers, the general identity `euler_formula_of_sphere2`. The tetrahedron incidence structure is then Platonic (`tetrahedron_platonic`). Combining Euler–Poincaré with regularity incidence gives the classification from homology (`platonic_solids_3d_of_homology`).

2.2. Incidence Combinatorics

The Lean gist is `PlatonicSolids/Incidence.lean` (with the pair predicate in `PlatonicSolids/PlatonicPair.lean`). A regular polyhedron has Schläfli symbol $\{p, q\}$, meaning: 1. Every 2D face is an identical regular p -gon ($p \geq 3$). Each face contributes p edge-face incidences. Since every edge is shared by exactly 2 faces:

$$pF = 2E \implies F = \frac{2E}{p}$$

2. Every vertex meets q faces and q edges ($q \geq 3$). Each vertex contributes q edge-vertex incidences. Since every edge connects exactly 2 vertices:

$$qV = 2E \implies V = \frac{2E}{q}$$

2.3 2.3. The Diophantine Reduction

The Lean gist is `PlatonicSolids/PlatonicPair.lean`; the reciprocal form $1/p + 1/q > 1/2$ is `PlatonicSolids/Reciprocal.lean`. Substituting these incidence relations into Euler’s formula:

$$\frac{2E}{q} - E + \frac{2E}{p} = 2$$

Multiplying by pq over $\mathbb{N}$ avoids rational arithmetic:

$$(2p + 2q)E = (pq)E + 2pq$$

Rearranging terms:

$$(2p + 2q - pq)E = 2pq$$

Because $2pq > 0$ for $p, q \geq 3$, the factor $(2p + 2q - pq)$ must be strictly positive, which yields:

$$pq < 2p + 2q \iff (p - 2)(q - 2) < 4$$

Since $p \geq 3$ and $q \geq 3$, both $(p - 2)$ and $(q - 2)$ are strictly positive integers. The only pairs of positive integers with product strictly less than 4 are:

$(p - 2)$	$(q - 2)$	$(p - 2)(q - 2)$	Pair (p, q)	Polyhedron
1	1	1	(3, 3)	Tetrahedron
1	2	2	(3, 4)	Octahedron
2	1	2	(4, 3)	Cube (Hexahedron)
1	3	3	(3, 5)	Icosahedron
3	1	3	(5, 3)	Dodecahedron

2.4 2.4. Formalization Insight: Elimination of the Non-Degeneracy Axiom

The Lean gist is `edges_pos_of_regular` in `PlatonicSolids/Incidence.lean`. In informal presentations, dividing Euler’s equation by $2E$ to obtain:

$$\frac{1}{p} + \frac{1}{q} = \frac{1}{2} + \frac{1}{E} > \frac{1}{2}$$

is typically justified by appealing to physical or geometric intuition: “*a solid polyhedron clearly has edges, so $E > 0$.*”

When formalizing this theorem in Lean 4, one might initially expect to require an explicit non-degeneracy hypothesis (`hE : E > 0`). However, machine verification reveals this hypothesis to be mathematically redundant.

Recall the cleared identity over $\mathbb{N}$:

$$(2p + 2q)E = (pq)E + 2pq$$

If $E = 0$, the equation reduces to:

$$0 = 2pq$$

Because $p \geq 3$ and $q \geq 3$, we have $2pq \geq 18$, which contradicts $0 = 2pq$.

Therefore, **the topological Euler characteristic ($\chi = 2$) together with the local regularity conditions ($p, q \geq 3$) algebraically prohibits an edgeless configuration.** The condition $E \geq 1$ is an algebraic consequence of the system, not an extrinsic geometric requirement.

3 3. Dimension 4: Why Topology Fails and Coxeter Takes Over

3.1 3.1. The Topological Barrier: $\chi(S^3) = 0$

The Lean gists are `PlatonicSolids/EulerPoincare.lean` (now also the 3-complex identity `euler_poincare3`), `PlatonicSolids/Sphere3Homology.lean` (cellular $H_*(S^3; \mathbb{Q})$ of the 4-simplex boundary), and `eulerChar_sphere3_betti` in `PlatonicSolids/EulerBetti.lean`.

A convex 4-polytope with nonempty interior has frontier homeomorphic to S^3 by the same radial projection as in §2.1. The standard simplicial 3-sphere is the boundary of a 4-simplex: 5 vertices, 10 edges, 10 triangles, 5 tetrahedra. Oriented incidence matrices $d_1 : \mathbb{Q}^{10} \rightarrow \mathbb{Q}^5$, $d_2 : \mathbb{Q}^{10} \rightarrow \mathbb{Q}^{10}$, $d_3 : \mathbb{Q}^5 \rightarrow \mathbb{Q}^{10}$ satisfy $d_1 d_2 = 0$ and $d_2 d_3 = 0$. Identity minors give $\text{rank } d_1 \geq 4$ and $\text{rank } d_3 \geq 4$; a 6×6 minor of d_2 has determinant 1, so $\text{rank } d_2 \geq 6$. The product-zero inequalities on the two middle dimensions of size 10 then force the ranks to be exactly 4, 6, and 4. Hence

$$b_0 = 5 - 4 = 1, \quad b_1 = (10 - 4) - 6 = 0, \quad b_2 = (10 - 6) - 4 = 0, \quad b_3 = 5 - 4 = 1$$

(`homology_sphere3`). The Euler characteristic vanishes:

$$\chi(S^3) = b_0 - b_1 + b_2 - b_3 = 1 - 0 + 0 - 1 = 0$$

Algebraic Euler–Poincaré in dimension 3 (`euler_poincare3`) again equates cell ranks with Betti ranks, so any such cellulation satisfies

$$V - E + F - C = 0$$

(`eulerChar_sphere3` on the 4-simplex, and `euler_formula_of_sphere3` in general). Because the alternating sum is zero, dividing by any flag count yields an identity equal to zero rather than a positive reciprocal like $2/E$. **Topology alone does not impose an upper bound on p, q , or r in 4D.**

3.2 3.2. Coxeter Systems and the Schläfli Gram Matrix

The Lean gist is `PlatonicSolids/GramMatrix.lean`. To obtain an upper bound, we use the theory of **Coxeter reflection groups**. A regular 4-polytope is described by its Schläfli symbol $\{p, q, r\}$: * The 3D cells are Platonic solids $\{p, q\}$. * The 3D vertex figures are Platonic solids $\{q, r\}$. * r cells meet around every 1D edge.

The Coxeter group W associated with $\{p, q, r\}$ is generated by reflections $\{s_1, s_2, s_3, s_4\}$ across the facet hyperplanes of a spherical fundamental simplex:

$$W = \langle s_1, s_2, s_3, s_4 \mid (s_i s_j)^{m_{ij}} = 1 \rangle$$

where the exponents are $m_{ii} = 1$, $m_{12} = p$, $m_{23} = q$, $m_{34} = r$, and $m_{ij} = 2$ for $|i - j| > 1$.

The canonical bilinear form of this system is represented by the **Schläfli Gram matrix**:

$$M = \begin{pmatrix} 1 & -\cos \frac{\pi}{p} & 0 & 0 \\ -\cos \frac{\pi}{p} & 1 & -\cos \frac{\pi}{q} & 0 \\ 0 & -\cos \frac{\pi}{q} & 1 & -\cos \frac{\pi}{r} \\ 0 & 0 & -\cos \frac{\pi}{r} & 1 \end{pmatrix}$$

3.3 3.3. Sylvester’s Criterion and Principal Minors

The Lean gists are `PlatonicSolids/Angles.lean`, `LeadingMinors.lean`, `TrailingMinor.lean`, `D3IfPlatonic.lean`, `QuadForm.lean`, `CompletingSquare.lean`, and `Sylvester.lean`. By the Cartan–Killing–Coxeter classification: 1. M is **positive definite** ($\det(M) > 0$) $\iff W$ is

a finite reflection group acting on S^3 (a finite regular convex 4-polytope). 2. M is positive semidefinite with nullity 1 ($\det(M) = 0$) $\iff$ W is an affine reflection group (a Euclidean 3D space-filling honeycomb). 3. M has negative eigenvalues ($\det(M) < 0$) $\iff$ W is a hyperbolic reflection group (a honeycomb in hyperbolic space $\mathbb{H}^3$).

By Sylvester's criterion, M is positive definite if and only if all leading principal minors $D_k = \det(M_k)$ are strictly positive: * $D_1 = 1 > 0$. * $D_2 = 1 - \cos^2 \frac{\pi}{p} = \sin^2 \frac{\pi}{p} > 0$ (satisfied for all $p \geq 3$). * $D_3 = \sin^2 \frac{\pi}{p} - \cos^2 \frac{\pi}{q}$.

$$D_3 > 0 \iff \sin \frac{\pi}{p} > \cos \frac{\pi}{q} = \sin \left(\frac{\pi}{2} - \frac{\pi}{q} \right) \iff \frac{1}{p} + \frac{1}{q} > \frac{1}{2}$$

Thus, $D_3 > 0$ requires the cell $\{p, q\}$ to be a Platonic solid. * Symmetrically, the trailing 3×3 minor requires:

$$\frac{1}{q} + \frac{1}{r} > \frac{1}{2}$$

Thus, the vertex figure $\{q, r\}$ must also be a Platonic solid.

3.4 The Critical Dimension-4 Minor: D_4

The Lean gist is `PlatonicSolids/DetFormula.lean`. Expanding $D_4 = \det(M)$ along the fourth row:

$$D_4 = D_3 - \cos^2 \left(\frac{\pi}{r} \right) D_2 = \left(\sin^2 \frac{\pi}{p} - \cos^2 \frac{\pi}{q} \right) - \cos^2 \left(\frac{\pi}{r} \right) \sin^2 \left(\frac{\pi}{p} \right)$$

Factoring $\sin^2 \frac{\pi}{p}$:

$$D_4 = \sin^2 \left(\frac{\pi}{p} \right) \left(1 - \cos^2 \frac{\pi}{r} \right) - \cos^2 \left(\frac{\pi}{q} \right) = \sin^2 \left(\frac{\pi}{p} \right) \sin^2 \left(\frac{\pi}{r} \right) - \cos^2 \left(\frac{\pi}{q} \right)$$

Therefore, the condition that $\{p, q, r\}$ forms a finite regular convex 4-polytope is:

$$\Delta(p, q, r) := \sin^2 \left(\frac{\pi}{p} \right) \sin^2 \left(\frac{\pi}{r} \right) - \cos^2 \left(\frac{\pi}{q} \right) > 0$$

4 Evaluation of the 11 Candidate Triples

The Lean gists are `PlatonicSolids/CompatibleTriples.lean` (the eleven triples), `TrigValues.lean` (exact $\sin^2$ and $\cos^2$), `SignTable.lean` (sign of Δ), `Regular4.lean` (the six), and `Classification.lean` (the capstone).

Because both $\{p, q\}$ and $\{q, r\}$ must be Platonic, the indices must satisfy $p, q, r \in \{3, 4, 5\}$. Among the $5 \times 5 = 25$ combinations, only **11 triples** have a compatible middle index q .

The exact squared trigonometric values are:

$$\begin{aligned} \cos^2 \frac{\pi}{3} &= \frac{1}{4}, & \sin^2 \frac{\pi}{3} &= \frac{3}{4} \\ \cos^2 \frac{\pi}{4} &= \frac{1}{2}, & \sin^2 \frac{\pi}{4} &= \frac{1}{2} \\ \cos^2 \frac{\pi}{5} &= \frac{3 + \sqrt{5}}{8}, & \sin^2 \frac{\pi}{5} &= \frac{5 - \sqrt{5}}{8} \end{aligned}$$

Evaluating $\Delta(p, q, r)$ across all 11 candidate triples:

4.1 Group I: Spherical Polychora ($\Delta > 0$)

1. $\{3, 3, 3\}$ (5-cell / Simplex):

$$\Delta = \binom{3}{4} \binom{3}{4} - \frac{1}{4} = \frac{9}{16} - \frac{4}{16} = \frac{5}{16} > 0$$

2. $\{3, 3, 4\}$ (16-cell) & $\{4, 3, 3\}$ (8-cell / Tesseract):

$$\Delta = \binom{3}{4} \binom{1}{2} - \frac{1}{4} = \frac{3}{8} - \frac{2}{8} = \frac{1}{8} > 0$$

3. $\{3, 4, 3\}$ (24-cell — Exceptional Self-Dual Polytope):

$$\Delta = \binom{3}{4} \binom{3}{4} - \frac{1}{2} = \frac{9}{16} - \frac{8}{16} = \frac{1}{16} > 0$$

4. $\{3, 3, 5\}$ (600-cell) & $\{5, 3, 3\}$ (120-cell):

$$\Delta = \binom{3}{4} \binom{5 - \sqrt{5}}{8} - \frac{1}{4} = \frac{7 - 3\sqrt{5}}{32}$$

Since $7^2 = 49 > (3\sqrt{5})^2 = 45$, we have $7 - 3\sqrt{5} > 0$, so $\Delta > 0$.

4.2 Group II: Euclidean Honeycomb ($\Delta = 0$)

5. $\{4, 3, 4\}$ (Cubic Honeycomb of $\mathbb{R}^3$):

$$\Delta = \binom{1}{2} \binom{1}{2} - \frac{1}{4} = \frac{1}{4} - \frac{1}{4} = 0 \quad (\text{Affine / flat space-filling})$$

4.3 Group III: Hyperbolic Honeycombs ($\Delta < 0$)

6. $\{3, 5, 3\}$ (Icosacell Honeycomb):

$$\Delta = \binom{3}{4} \binom{3}{4} - \frac{3 + \sqrt{5}}{8} = \frac{3 - 2\sqrt{5}}{16} < 0 \quad (\text{since } 3^2 = 9 < 20)$$

7. $\{4, 3, 5\}$ & $\{5, 3, 4\}$:

$$\Delta = \binom{1}{2} \binom{5 - \sqrt{5}}{8} - \frac{1}{4} = \frac{1 - \sqrt{5}}{16} < 0$$

8. $\{5, 3, 5\}$:

$$\Delta = \left(\frac{5 - \sqrt{5}}{8} \right)^2 - \frac{1}{4} = \frac{14 - 10\sqrt{5}}{64} < 0 \quad (\text{since } 14^2 = 196 < 500)$$

Thus, precisely 6 triples yield positive-definite Gram matrices.

5 5. Complete Lean 4 Formal Verification

The Lake package compiles against Lean 4 and Mathlib. It contains **zero axioms, zero admits, and zero sorrys** outside the deliberate holes in `Challenge.lean`. The development is split into section-sized modules under `PlatonicSolids/`, each meant to be copied as a gist next to the matching paragraph above. `PlatonicSolids.lean` is the root import. The full sources are reproduced in the appendix.

The compared theorems are:

- `platonic_solids_3d` — Euler $V - E + F = 2$ plus regularity incidence $pF = 2E$, $qV = 2E$ force a Platonic pair (p, q) .
- `edges_pos_of_regular` — the same identities algebraically force $E > 0$; non-degeneracy is a theorem, not an axiom.
- `regular_polychora_classification` — Platonic cells and vertex figures with $\Delta(p, q, r) > 0$ are exactly the six regular convex 4-polytopes.

The stronger equivalence `regular_polychora_iff` identifies those six with positive-definiteness of `schlafligram` for $p, q, r \geq 3$, via Sylvester’s criterion (`PlatonicSolids/Sylvester.lean`).

`Challenge.lean` restates the compared family with deliberate **sorrys** so Palomar Comparator can check types against `Solution.lean`. Supporting definitions are `IsPlatonicPair`, `IsRegular4Polytope`, `schlafligram`, and `schlafliDet`.

File	Paper	Content
<code>PlatonicSolids/RadialProjection.lean</code>	§2.1	Frontier of a convex body $\simeq$ sphere
<code>PlatonicSolids/EulerPoincare.lean</code>	§§2.1, 3.1	Algebraic Euler–Poincaré in dimensions 2 and 3
<code>PlatonicSolids/SingularHomology.lean</code>	§2.1	Homotopy invariance; $H_0(S^n)$; $H_*(S^0)$
<code>PlatonicSolids/RelativeHomology.lean</code>	§2.1	Pair sequence $0 \rightarrow C_*(A) \rightarrow C_*(X) \rightarrow C_*(X, A) \rightarrow 0$ and LES
<code>PlatonicSolids/MayerVietoris.lean</code>	§2.1	Triad SES, excision predicate, stereographic cover of S^n
<code>PlatonicSolids/Sphere2Homology.lean</code>	§2.1	Cellular $H_*(S^2; \mathbb{Q})$ of the tetrahedron surface
<code>PlatonicSolids/Sphere3Homology.lean</code>	§3.1	Cellular $H_*(S^3; \mathbb{Q})$ of the 4-simplex boundary
<code>PlatonicSolids/EulerBetti.lean</code>	§§2.1, 3.1	$\chi(S^2) = 2$, $\chi(S^3) = 0$
<code>PlatonicSolids/PlatonicPair.lean</code>	§2.3	Five solutions of $(p-2)(q-2) < 4$
<code>PlatonicSolids/Reciprocal.lean</code>	§§2.3, 3.3	$1/p + 1/q > 1/2$
<code>PlatonicSolids/Incidence.lean</code>	§§2.2–2.4	<code>platonic_solids_3d</code> , <code>edges_pos_of_regular</code>
<code>PlatonicSolids/Angles.lean</code>	§3.3	$\pi/n \in (0, \pi/2)$
<code>PlatonicSolids/GramMatrix.lean</code>	§3.2	<code>schlafligram</code> , <code>schlafliDet</code>

File	Paper	Content
PlatonicSolids/ LeadingMinors.lean	§3.3	D_1, D_2, D_3
PlatonicSolids/ TrailingMinor.lean	§3.3	Vertex-figure 3×3
PlatonicSolids/ D3IffPlatonic.lean	§3.3	$D_3 > 0$ iff Platonic cell
PlatonicSolids/ QuadForm.lean	§3.3	Gram quadratic form
PlatonicSolids/ CompletingSquare.lean	§3.3	LDL identity
PlatonicSolids/ DetFormula.lean	§3.4	$\det M = \Delta$
PlatonicSolids/ Sylvester.lean	§3.3	PosDef iff leading minors > 0
PlatonicSolids/ TrigValues.lean	§4	$\sin^2(\pi/n), \cos^2(\pi/n)$ for $n \in \{3, 4, 5\}$
PlatonicSolids/ CompatibleTriples.lean	§4	Eleven compatible triples
PlatonicSolids/ SignTable.lean	§4	Sign of Δ
PlatonicSolids/ Regular4.lean	§4	The six regular 4-polytopes
PlatonicSolids/ Classification.lean	§4	Compared 4D theorem and PosDef iff

The remaining named lemmas are the computational steps of those paragraphs: `platonic_pairs_of_inequality` / `platonic_pair_iff` / `platonic_pair_iff_reciprocal` for the Diofantine pair; `leadingMinor_one/two/three` and `leadingMinor_three_pos_iff` / `trailing3_pos_iff` for Sylvester's first three minors; `schlafligram_quad` / `band_quad_squares` / `schlafligram_det` for $\det M = \Delta$; `schlafliDet_*` and `schlafliDet_*_pos/neg` for the eleven signs; `regular_polychora_iff` / `posDef_of_regular` / `regular_of_posDef` for the PosDef equivalence.

6 6. Pedagogical and Formalization Significance

1. **Foundational Consistency Across Dimensions:** The mathematical transition between 3D and 4D is made explicit: 3D regular polytopes are bounded by the topology of S^2 , whereas 4D polychora are bounded by the spectral properties of Coxeter reflection groups via Sylvester's criterion.
2. **Elimination of Redundant Hypotheses:** As demonstrated in `edges_pos_of_regular`, Lean's linter and tactic engine show that the geometric non-degeneracy condition $E > 0$ is a theorem rather than an independent axiom.
3. **Absence of Ad-Hoc Combinatorial Filters:** Candidate 4D triples are filtered using the **Schläfli determinant** $\Delta(p, q, r) = \sin^2 \frac{\pi}{p} \sin^2 \frac{\pi}{r} - \cos^2 \frac{\pi}{q}$. Every rejection is certified by non-linear arithmetic over real algebraic numbers, with all properties of $\sqrt{5}$ derived from first principles.

7 Acknowledgements

The author expresses sincere gratitude to Professor Dana Scott (Carnegie Mellon University) for suggesting this topic and for inspiring discussions on the formalization of polyhedral geometry. Any remaining errors, misinterpretations, or omissions are solely the author’s own.

7.1 Declaration of Generative AI and AI-Assisted Technologies

In accordance with the Committee on Publication Ethics (COPE) position statements and guidelines on artificial intelligence in research and formal authorship, the author acknowledges the use of large language model tooling during the preparation of this work. Specifically, **Google Gemini 3.8** and **Cursor Grok 4.6** were employed as interactive coding and drafting assistants to assist in exploratory formalization, synthesize initial Lean 4 proof tactics, and help structure the expository drafts. All mathematical claims, Lean 4 proofs, and final text were subsequently reviewed, verified, refined, and certified by the author, who assumes full responsibility for the integrity and accuracy of the content.

8 References

1. Coxeter, H. S. M. *Regular Polytopes*. 3rd ed., Dover Publications, 1973.
2. Schläfli, L. *Theorie der vielfachen Kontinuität*. Zürcher & Furrer, 1852.
3. Euclid. *The Thirteen Books of the Elements*, Vol. 3 (Books X–XIII). Translated by T. L. Heath, Cambridge University Press, 1908.
4. Humphreys, J. E. *Reflection Groups and Coxeter Groups*. Cambridge Studies in Advanced Mathematics, Cambridge University Press, 1990.
5. The Mathlib Community. *The Lean 4 Mathematical Library*. <https://github.com/leanprover-community/mathlib4>
6. Committee on Publication Ethics (COPE). *Authorship and AI Tools: COPE Position Statement*. <https://publicationethics.org/guidance/guidelines/authorship-and-ai-tools>
7. Honner, P. “There Are Just Five Perfect Shapes in Math—Here’s Why.” *Scientific American*, 2023. <https://www.scientificamerican.com/article/there-are-just-five-perfect-shapes-in-math-heres-why/>
8. Thought Thrill. “The Five Platonic Solids and Why There Can Only Be Five.” YouTube video, 2026.
9. Quick, M. “Regular 4-Polytopes (Polychora) Overview and Interactive Visualizations.” *Quick’s 4D Space*. <https://www.qfbox.info/4d/index>

A Complete Lean source

Checked by `lake build` against Lean 4 and Mathlib. The development contains no `sorry`, no `admit`, and no project-defined axiom. Palomar Comparator compares `platonic_solids_3d`, `edges_pos_of_regular`, and `regular_polychora_classification`.

A.1 PlatonicSolids.lean

Root import of the section-sized modules (37 lines).

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
Github: https://github.com/catskillsresearch/platonic-solids/blob/main/PlatonicSolids.lean
-/
import PlatonicSolids.Angles
import PlatonicSolids.Classification
import PlatonicSolids.CompatibleTriples
import PlatonicSolids.CompletingSquare
import PlatonicSolids.D3IffPlatonic
import PlatonicSolids.DetFormula
import PlatonicSolids.EulerBetti
import PlatonicSolids.EulerPoincare
import PlatonicSolids.GramMatrix
import PlatonicSolids.Incidence
import PlatonicSolids.LeaningMinors
import PlatonicSolids.MayerVietoris
import PlatonicSolids.PlatonicPair
import PlatonicSolids.QuadForm
import PlatonicSolids.RadialProjection
import PlatonicSolids.Reciprocal
import PlatonicSolids.Regular4
import PlatonicSolids.RelativeHomology
import PlatonicSolids.SignTable
import PlatonicSolids.SingularHomology
import PlatonicSolids.Sphere2Homology
import PlatonicSolids.Sphere3Homology
import PlatonicSolids.Sylvester
import PlatonicSolids.TrailingMinor
import PlatonicSolids.TrigValues

/-!
Root import of the section-sized modules. Each file under `PlatonicSolids/`
is meant to be readable next to the corresponding paragraph of
`PlatonicSolids.md`.
-/
```

A.2 PlatonicSolids/RadialProjection.lean

Frontier of a bounded convex body is homeomorphic to the sphere (25 lines).

```
/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.Convex.GaugeRescale

/-!
Paper §2.1. Radial projection: the frontier of a bounded convex body
with nonempty interior is homeomorphic to the unit sphere. Mathlib's
`gaugeRescaleHomeomorph` is the homeomorphism of the ambient space
that carries the frontier onto `Metric.sphere 0 1`.
-/

open Metric Set

/-- The frontier of a bounded convex set with nonempty interior is a
topological sphere. This is the radial projection of the paper, stated
```

```

for a body that need not be centred at the origin. -/
theorem frontier_convex_homeo_sphere
  {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {s : Set E}
  (hc : Convex ℝ s) (hne : (interior s).Nonempty) (hb : Bornology.IsBounded s) :
  ∃ h : E ≃t E, h '' frontier s = sphere (0 : E) 1 :=
  let ⟨h, _, _, hf⟩ :=
    exists_homeomorph_image_interior_closure_frontier_eq_unitBall hc hne hb
  ⟨h, hf⟩

```

A.3 PlatonicSolids/EulerPoincare.lean

Algebraic Euler–Poincaré in dimensions 2 and 3 (112 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.FiniteDimensional.Lemmas

/-!
Paper §2.1. Algebraic Euler–Poincaré for a 2-dimensional chain complex
of finite-dimensional vector spaces (the cellular complex of a polyhedron):

 $\chi(C) = \text{rank } C_0 - \text{rank } C_1 + \text{rank } C_2$  equals  $\chi(H) = b_0 - b_1 + b_2$ .

The identity is rank-nullity in dimensions 2 and 3. Specializing
to the Betti numbers of the tetrahedron surface
(`PlatonicSolids/Sphere2Homology.lean`) yields  $V + F = E + 2$ .
Specializing to the 4-simplex boundary
(`PlatonicSolids/Sphere3Homology.lean`) yields  $V - E + F - C = 0$ .
-/

open Module

variable {K : Type*} [Field K]
variable {C2 C1 C0 : Type*}
  [AddCommGroup C2] [Module K C2] [FiniteDimensional K C2]
  [AddCommGroup C1] [Module K C1] [FiniteDimensional K C1]
  [AddCommGroup C0] [Module K C0]

/--  $b_0 = \text{rank}(C_0 / \text{im } d_1)$ . -/
noncomputable def betti0 (d1 : C1 →1[K] C0) : ℤ :=
  finrank K C0 - finrank K (LinearMap.range d1)

/--  $b_1 = \text{rank}(\ker d_1 / \text{im } d_2)$ . -/
noncomputable def betti1 (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0) : ℤ :=
  finrank K (LinearMap.ker d1) - finrank K (LinearMap.range d2)

/--  $b_2 = \text{rank}(\ker d_2)$ . -/
noncomputable def betti2 (d2 : C2 →1[K] C1) : ℤ :=
  finrank K (LinearMap.ker d2)

/-- Algebraic Euler–Poincaré: cell Euler characteristic equals homological
Euler characteristic for any 2-complex with  $d_1 \circ d_2 = 0$ . -/
theorem euler_poincare (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  (hcomp : d1 ∘ d2 = 0) :
  (finrank K C0 : ℤ) - finrank K C1 + finrank K C2 =
    betti0 d1 - betti1 d2 d1 + betti2 d2 := by
  have _hB : LinearMap.range d2 ≤ LinearMap.ker d1 :=
    LinearMap.range_le_ker_iff.mpr hcomp
  have hr1 : (finrank K (LinearMap.range d1) : ℤ) + finrank K (LinearMap.ker d1) =
    finrank K C1 := by exact_mod_cast d1.finrank_range_add_finrank_ker
  have hr2 : (finrank K (LinearMap.range d2) : ℤ) + finrank K (LinearMap.ker d2) =

```

```

    finrank K C2 := by exact_mod_cast d2.finrank_range_add_finrank_ker
simp only [betti0, betti1, betti2]
linarith

/-- `V - E + F = 2` is the same counting identity as `V + F = E + 2`. -/
theorem euler_count {V E F : ℕ} (h : (V : ℤ) - E + F = 2) : V + F = E + 2 := by
  omega

/-- Cellular ranks `V, E, F` and the Betti numbers of `S2` give Euler's formula. -/
theorem euler_formula_of_sphere2
  (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  {V E F : ℕ}
  (hV : finrank K C0 = V) (hE : finrank K C1 = E) (hF : finrank K C2 = F)
  (hcomp : d1.comp d2 = 0)
  (hb0 : betti0 d1 = 1) (hb1 : betti1 d2 d1 = 0) (hb2 : betti2 d2 = 1) :
  V + F = E + 2 := by
  have hχ := euler_poincare d2 d1 hcomp
  rw [hV, hE, hF, hb0, hb1, hb2] at hχ
  exact euler_count hχ

variable {C3 : Type*}
  [AddCommGroup C3] [Module K C3] [FiniteDimensional K C3]

/-- In a 3-complex, `b2 = rank(ker d2 / im d3)`. -/
noncomputable def betti2Rel (d3 : C3 →1[K] C2) (d2 : C2 →1[K] C1) : ℤ :=
  finrank K (LinearMap.ker d2) - finrank K (LinearMap.range d3)

/-- `b3 = rank(ker d3)`. -/
noncomputable def betti3 (d3 : C3 →1[K] C2) : ℤ :=
  finrank K (LinearMap.ker d3)

/-- Algebraic Euler-Poincaré in dimension 3. -/
theorem euler_poincare3 (d3 : C3 →1[K] C2) (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  (h32 : d2.comp d3 = 0) (h21 : d1.comp d2 = 0) :
  (finrank K C0 : ℤ) - finrank K C1 + finrank K C2 - finrank K C3 =
    betti0 d1 - betti1 d2 d1 + betti2Rel d3 d2 - betti3 d3 := by
  have _hB21 : LinearMap.range d2 ≤ LinearMap.ker d1 :=
    LinearMap.range_le_ker_iff.mpr h21
  have _hB32 : LinearMap.range d3 ≤ LinearMap.ker d2 :=
    LinearMap.range_le_ker_iff.mpr h32
  have hr1 : (finrank K (LinearMap.range d1) : ℤ) + finrank K (LinearMap.ker d1) =
    finrank K C1 := by exact_mod_cast d1.finrank_range_add_finrank_ker
  have hr2 : (finrank K (LinearMap.range d2) : ℤ) + finrank K (LinearMap.ker d2) =
    finrank K C2 := by exact_mod_cast d2.finrank_range_add_finrank_ker
  have hr3 : (finrank K (LinearMap.range d3) : ℤ) + finrank K (LinearMap.ker d3) =
    finrank K C3 := by exact_mod_cast d3.finrank_range_add_finrank_ker
  simp only [betti0, betti1, betti2Rel, betti3]
  linarith

/-- Cell counts of an `S3` complex with Betti numbers `1,0,0,1` give `χ = 0`. -/
theorem euler_formula_of_sphere3
  (d3 : C3 →1[K] C2) (d2 : C2 →1[K] C1) (d1 : C1 →1[K] C0)
  {V E F C : ℕ}
  (hV : finrank K C0 = V) (hE : finrank K C1 = E)
  (hF : finrank K C2 = F) (hC : finrank K C3 = C)
  (h32 : d2.comp d3 = 0) (h21 : d1.comp d2 = 0)
  (hb0 : betti0 d1 = 1) (hb1 : betti1 d2 d1 = 0)
  (hb2 : betti2Rel d3 d2 = 0) (hb3 : betti3 d3 = 1) :
  (V : ℤ) - E + F - C = 0 := by
  have hχ := euler_poincare3 d3 d2 d1 h32 h21
  rw [hV, hE, hF, hC, hb0, hb1, hb2, hb3] at hχ
  linarith

```

A.4 PlatonicSolids/SingularHomology.lean

Singular $H_0(S^n)$, $H_*(S^0)$, homotopy invariance (156 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.AlgebraicTopology.SingularHomology.HomologyZero
import Mathlib.AlgebraicTopology.SingularHomology.HomotopyInvariance
import Mathlib.Analysis.InnerProductSpace.PiL2
import Mathlib.Analysis.Normed.Module.Connected
import Mathlib.LinearAlgebra.Dimension.StrongRankCondition
import Mathlib.Topology.Homotopy.Contractible
import Mathlib.Topology.Homotopy.Equiv
import Mathlib.Topology.Separation.Lemmas

/-!
Singular homology lemmas that Mathlib does not package: homotopy
equivalences and homeomorphisms induce homology isomorphisms,
contractible spaces have the homology of a point,  $H_0(S^n) \cong \mathbb{R}$  for
 $n \geq 1$ , and the full calculation of  $H_*(S^0)$ .

The pair and Mayer-Vietoris sequences of singular chains are in
`PlatonicSolids/RelativeHomology.lean` and
`PlatonicSolids/MayerVietoris.lean`. Excision for the stereographic
cover of `Metric.sphere` (small simplices) is still absent, so this
file does not identify  $H_n(S^n)$  for  $n > 0$ . The paper's  $\chi(S^3) = 0$ 
is obtained from the cellular calculation in
`PlatonicSolids/Sphere3Homology.lean`.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex
open scoped ContinuousMap Topology

universe v u

variable {C : Type u} [Category.{v} C] [HasCoproducts.{0} C] [Preadditive C]
[CategoryWithHomology C]

/-- Homotopic continuous maps induce equal singular-homology maps. -/
lemma singularHomologyMap_eq_of_homotopic
  {X Y : TopCat.{0}} {f g : X → Y} (h : f.hom.Homotopic g.hom)
  (R : C) (n : ℕ) :
  homologyMap (((singularChainComplexFunctor C).obj R).map f) n =
  homologyMap (((singularChainComplexFunctor C).obj R).map g) n :=
  TopCat.Homotopy.congr_homologyMap_singularChainComplexFunctor h.some R n

/-- A homotopy equivalence of spaces induces a homotopy equivalence of
singular chain complexes. -/
noncomputable def singularChainHomotopyEquiv
  {X Y : Type} [TopologicalSpace X] [TopologicalSpace Y]
  (e : X ≃h Y) (R : C) :
  HomotopyEquiv
  (((singularChainComplexFunctor C).obj R).obj (.of X))
  (((singularChainComplexFunctor C).obj R).obj (.of Y)) where
  hom := ((singularChainComplexFunctor C).obj R).map (TopCat.ofHom e.toFun)
  inv := ((singularChainComplexFunctor C).obj R).map (TopCat.ofHom e.invFun)
  homotopyHomInvId := by
  have H : TopCat.Homotopy
    (TopCat.ofHom e.toFun >> TopCat.ofHom e.invFun) (⊥) := by
    simpa [TopCat.ofHom_comp, TopCat.ofHom_id] using e.left_inv.some
    simpa [Functor.map_comp, Functor.map_id] using
      H.singularChainComplexFunctorObjMap R

```

```

homotopyInvHomId := by
  have H : TopCat.Homotopy
    (TopCat.ofHom e.invFun >> TopCat.ofHom e.toFun) (≠ _) := by
      simp [TopCat.ofHom_comp, TopCat.ofHom_id] using e.right_inv.some
      simp [Functor.map_comp, Functor.map_id] using
        H.singularChainComplexFunctorObjMap R

/-- Homotopy equivalent spaces have isomorphic singular homology. -/
noncomputable def singularHomologyIso_of_homotopyEquiv
  {X Y : Type} [TopologicalSpace X] [TopologicalSpace Y]
  (e : X ≃h Y) (R : C) (n : ℕ) :
  ((singularHomologyFunctor C n).obj R).obj (.of X) ≅
    ((singularHomologyFunctor C n).obj R).obj (.of Y) :=
  (singularChainHomotopyEquiv e R).toHomologyIso n

/-- Homeomorphic spaces have isomorphic singular homology. -/
noncomputable def singularHomologyIso_of_homeomorph
  {X Y : Type} [TopologicalSpace X] [TopologicalSpace Y]
  (e : X ≃t Y) (R : C) (n : ℕ) :
  ((singularHomologyFunctor C n).obj R).obj (.of X) ≅
    ((singularHomologyFunctor C n).obj R).obj (.of Y) :=
  singularHomologyIso_of_homotopyEquiv e.toHomotopyEquiv R n

/-- A point has vanishing singular homology in positive degree. -/
lemma isZero_singularHomology_unit (R : C) {n : ℕ} (hn : n ≠ 0) :
  IsZero (((singularHomologyFunctor C n).obj R).obj (.of Unit)) :=
  isZero_singularHomologyFunctor_of_totallyDisconnectedSpace C n R (.of Unit) hn

/-- A contractible space has vanishing singular homology in positive degree. -/
lemma isZero_singularHomology_of_contractible
  {X : Type} [TopologicalSpace X] [ContractibleSpace X]
  (R : C) {n : ℕ} (hn : n ≠ 0) :
  IsZero (((singularHomologyFunctor C n).obj R).obj (.of X)) := by
  obtain ⟨e⟩ := ContractibleSpace.hequiv_unit X
  exact (isZero_singularHomology_unit R hn).of_iso
    (singularHomologyIso_of_homotopyEquiv e R n)

/-- Path-connected spaces have  $H_0 \cong R$  via the augmentation. -/
noncomputable def singularHomology0Iso_pathConnected
  (X : TopCat.{0}) [PathConnectedSpace X] (R : C) :
  ((singularHomologyFunctor C 0).obj R).obj X ≅ R :=
  asIso (X.singularHomology0 R)

/-- The unit sphere in Euclidean dimension  $n+1 \geq 2$  is path-connected. -/
instance pathConnectedSpace_sphere_euclidean
  {n : ℕ} [NeZero n] :
  PathConnectedSpace (↑(Metric.sphere (0 : EuclideanSpace ℝ (Fin (n + 1))) 1)) := by
  rw [← isPathConnected_iff_pathConnectedSpace]
  refine isPathConnected_sphere ?_ 0 (by norm_num)
  have : (1 : ℕ) < n + 1 := Nat.lt_add_of_pos_left (NeZero.pos n)
  have hfin : Module.finrank ℝ (EuclideanSpace ℝ (Fin (n + 1))) = n + 1 := by
    simp
  rw [← Module.finrank_eq_rank, hfin]
  exact_mod_cast this

/--  $H_0(S^n; R) \cong R$  for  $n \geq 1$ . -/
noncomputable def singularHomology0_sphere
  {n : ℕ} [NeZero n] (R : C) :
  ((singularHomologyFunctor C 0).obj R).obj
    (.of ↑(Metric.sphere (0 : EuclideanSpace ℝ (Fin (n + 1))) 1)) ≅ R :=
  singularHomology0Iso_pathConnected _ R

/--  $S^0 \subset \mathbb{R}$  is the two-point set  $\{\pm 1\}$ . -/

```

```

lemma sphere_real_eq : (Metric.sphere (0 : ℝ) 1 : Set ℝ) = {1, -1} := by
  ext x
  simp [mem_sphere_iff_norm, Real.norm_eq_abs, abs_eq]

instance : TotallyDisconnectedSpace ↑(Metric.sphere (0 : ℝ) 1) := by
  refine totallyDisconnectedSpace_subtype_iff.2
    (Set.Countable.isTotallyDisconnected ?_)
  rw [sphere_real_eq]
  exact ((Set.finite_singleton (-1 : ℝ)).insert 1).countable

/-- `H_n(S^0; R) = 0` for `n > 0`. -/
lemma isZero_singularHomology_sphere0 (R : C) {n : ℕ} (hn : n ≠ 0) :
  IsZero (((singularHomologyFunctor C n).obj R).obj
    (.of ↑(Metric.sphere (0 : ℝ) 1))) :=
  isZero_singularHomologyFunctor_of_totallyDisconnectedSpace C n R _ hn

/-- `H_0(S^0; R) ≅ R ⊕ R`. -/
noncomputable def singularHomology_0_sphere0 (R : C) :
  ((singularHomologyFunctor C 0).obj R).obj (.of ↑(Metric.sphere (0 : ℝ) 1)) ≅
  ⨆ fun _ : ↑(Metric.sphere (0 : ℝ) 1) ↦ R :=
  singularHomologyFunctorZeroOfTotallyDisconnectedSpace C R
  (.of ↑(Metric.sphere (0 : ℝ) 1))

/-- The closed unit disk is contractible, hence has vanishing positive
singular homology. -/
lemma isZero_singularHomology_closedBall
  {E : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
  (R : C) {n : ℕ} (hn : n ≠ 0) :
  IsZero (((singularHomologyFunctor C n).obj R).obj
    (.of ↑(Metric.closedBall (0 : E) 1))) := by
  have : ContractibleSpace ↑(Metric.closedBall (0 : E) 1) :=
    Metric.contractibleSpace_closedBall (by norm_num)
  exact isZero_singularHomology_of_contractible R hn

```

A.5 PlatonicSolids/RelativeHomology.lean

Relative singular homology and the LES of a pair (147 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Homology.HomologicalComplexAbelian
import Mathlib.Algebra.Homology.HomologySequence
import Mathlib.AlgebraicTopology.SingularHomology.Basic
import Mathlib.Topology.Category.TopCat.EpiMono

/-!
Relative singular homology of a pair `(X, A)` as the cokernel of the
inclusion of singular chains. Mathlib's singular chain-complex functor
preserves monomorphisms (it is free on simplices), so a subspace
inclusion yields a short exact sequence of chain complexes and therefore
the long exact sequence of the pair.

This is the first half of the classical path to `H_n(S^n)`: the remaining
obstruction is excision / small simplices, packaged in
`PlatonicSolids/MayerVietoris.lean`.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex
open scoped Topology

set_option linter.unusedSectionVars false

```

```

universe v u

variable {C : Type u} [Category.{v} C] [Abelian C]
[HasCoproducts.{0} C] [HasPullbacks C]

variable (R : C)

/-- Inclusion of a subspace as a morphism of `TopCat`. -/
def subtypeIncl (X : Type) [TopologicalSpace X] (A : Set X) :
  TopCat.of  $\uparrow$ A  $\longrightarrow$  TopCat.of X :=
  TopCat.ofHom (Subtype.val, continuous_subtype_val)

instance subtypeIncl_mono (X : Type) [TopologicalSpace X] (A : Set X) :
  Mono (subtypeIncl X A) :=
  (TopCat.mono_iff_injective _).2 Subtype.val_injective

/-- Singular chains of a subspace, included into those of the ambient space. -/
noncomputable def inclChains (X : Type) [TopologicalSpace X] (A : Set X) :
  ((singularChainComplexFunctor C).obj R).obj (.of  $\uparrow$ A)  $\longrightarrow$ 
  ((singularChainComplexFunctor C).obj R).obj (.of X) :=
  ((singularChainComplexFunctor C).obj R).map (subtypeIncl X A)

instance inclChains_mono (X : Type) [TopologicalSpace X] (A : Set X) :
  Mono (inclChains (C := C) R X A) :=
  ((singularChainComplexFunctor C).obj R).map_mono (subtypeIncl X A)

/-- Relative singular chains  $C_*(X, A; R) := \text{coker}(C_*(A) \rightarrow C_*(X))$ . -/
noncomputable def relativeSingularChains (X : Type) [TopologicalSpace X] (A : Set X) :
  ChainComplex C  $\mathbb{N}$  :=
  cokernel (inclChains (C := C) R X A)

/-- The short complex  $C_*(A) \rightarrow C_*(X) \rightarrow C_*(X, A)$ . -/
noncomputable def pairShortComplex (X : Type) [TopologicalSpace X] (A : Set X) :
  ShortComplex (ChainComplex C  $\mathbb{N}$ ) :=
  ShortComplex.mk (inclChains (C := C) R X A) (cokernel. $\pi$  _) (cokernel.condition _)

instance pairShortComplex_mono_f (X : Type) [TopologicalSpace X] (A : Set X) :
  Mono (pairShortComplex (C := C) R X A).f :=
  inferInstanceAs (Mono (inclChains (C := C) R X A))

instance pairShortComplex_epi_g (X : Type) [TopologicalSpace X] (A : Set X) :
  Epi (pairShortComplex (C := C) R X A).g :=
  inferInstanceAs (Epi (cokernel. $\pi$  (inclChains (C := C) R X A)))

/-- The pair sequence of singular chains is short exact. -/
lemma pair_shortExact (X : Type) [TopologicalSpace X] (A : Set X) :
  (pairShortComplex (C := C) R X A).ShortExact :=
  ShortComplex.ShortExact.mk'
  (ShortComplex.exact_of_g_is_cokernel _
    (cokernelIsCokernel (inclChains (C := C) R X A)))
  inferInstance inferInstance

/-- Relative singular homology  $H_n(X, A; R)$ . -/
noncomputable def relativeSingularHomology
  (X : Type) [TopologicalSpace X] (A : Set X) (n :  $\mathbb{N}$ ) : C :=
  (relativeSingularChains (C := C) R X A).homology n

/-- The map  $H_n(A) \rightarrow H_n(X)$  induced by the subspace inclusion. -/
noncomputable def pair_i
  (X : Type) [TopologicalSpace X] (A : Set X) (n :  $\mathbb{N}$ ) :
  (((singularChainComplexFunctor C).obj R).obj (.of  $\uparrow$ A)).homology n  $\longrightarrow$ 
  (((singularChainComplexFunctor C).obj R).obj (.of X)).homology n :=

```

```

homologyMap (inclChains (C := C) R X A) n

/-- The map  $H_n(X) \rightarrow H_n(X, A)$  to relative homology. -/
noncomputable def pair_j
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    (((singularChainComplexFunctor C).obj R).obj (.of X)).homology n →
    relativeSingularHomology (C := C) R X A n :=
  homologyMap (cokernel.π (inclChains (C := C) R X A)) n

/-- Connecting homomorphism  $\delta : H_{n+1}(X, A) \rightarrow H_n(A)$  of the pair. -/
noncomputable def pair_δ
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    relativeSingularHomology (C := C) R X A (n + 1) →
    (((singularChainComplexFunctor C).obj R).obj (.of ↑A)).homology n :=
  (pair_shortExact (C := C) R X A).δ (n + 1) n (by simp)

/-- The composition  $H_n(A) \rightarrow H_n(X) \rightarrow H_n(X, A)$  is zero. -/
lemma pair_i_comp_pair_j
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    pair_i (C := C) R X A n >> pair_j (C := C) R X A n = 0 := by
  simpa [pair_i, pair_j] using
    (by
      rw [← homologyMap_comp, cokernel.condition, homologyMap_zero] :
      homologyMap (inclChains (C := C) R X A) n >>
      homologyMap (cokernel.π (inclChains (C := C) R X A)) n = 0)

/-- The composition  $H_n(X) \rightarrow H_n(X, A) \rightarrow H_{n-1}(A)$  is zero. -/
lemma pair_j_comp_pair_δ
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    pair_j (C := C) R X A (n + 1) >> pair_δ (C := C) R X A n = 0 :=
  (pair_shortExact (C := C) R X A).comp_δ (n + 1) n (by simp)

/-- The composition  $H_{n+1}(X, A) \rightarrow H_n(A) \rightarrow H_n(X)$  is zero. -/
lemma pair_δ_comp_pair_i
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    pair_δ (C := C) R X A n >> pair_i (C := C) R X A n = 0 :=
  (pair_shortExact (C := C) R X A).δ_comp (n + 1) n (by simp)

/-- Exactness of  $H_n(A) \rightarrow H_n(X) \rightarrow H_n(X, A)$ . -/
lemma pair_exact_at_X
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    (ShortComplex.mk (pair_i (C := C) R X A n) (pair_j (C := C) R X A n)
      (pair_i_comp_pair_j (C := C) R X A n)).Exact := by
  simpa [pair_i, pair_j, pairShortComplex] using
    (pair_shortExact (C := C) R X A).homology_exact2 n

/-- Exactness of  $H_n(X) \rightarrow H_n(X, A) \rightarrow H_{n-1}(A)$ . -/
lemma pair_exact_at_rel
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    (ShortComplex.mk (pair_j (C := C) R X A (n + 1)) (pair_δ (C := C) R X A n)
      (pair_j_comp_pair_δ (C := C) R X A n)).Exact :=
  (pair_shortExact (C := C) R X A).homology_exact3 (n + 1) n (by simp)

/-- Exactness of  $H_{n+1}(X, A) \rightarrow H_n(A) \rightarrow H_n(X)$ . -/
lemma pair_exact_at_A
  (X : Type) [TopologicalSpace X] (A : Set X) (n : ℕ) :
    (ShortComplex.mk (pair_δ (C := C) R X A n) (pair_i (C := C) R X A n)
      (pair_δ_comp_pair_i (C := C) R X A n)).Exact :=
  (pair_shortExact (C := C) R X A).homology_exact1 (n + 1) n (by simp)

```

A.6 PlatonicSolids/MayerVietoris.lean

Mayer–Vietoris of an excisive triad; stereographic cover of S^n (315 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Algebra.Homology.HomologicalComplexBiprod
import Mathlib.Algebra.Homology.QuasiIso
import Mathlib.Analysis.Convex.Contractible
import Mathlib.Analysis.InnerProductSpace.PiL2
import Mathlib.Geometry.Manifold.Instances.Sphere
import Mathlib.Topology.Homeomorph.Lemmas
import PlatonicSolids.RelativeHomology
import PlatonicSolids.SingularHomology

/-!
Mayer-Vietoris for a commuting square of spaces

...
      W --iU--> U
      |         |
      iV         jU
      |         |
      v         v
      V --jV--> X
...

The map  $C_*(W) \rightarrow C_*(U) \oplus C_*(V)$ ,  $\sigma \mapsto (iU_\# \sigma, -iV_\# \sigma)$  is a monomorphism
as soon as  $iU$  is. Its cokernel is the chain complex of the cover; the
commuting square induces a comparison  $coker \rightarrow C_*(X)$ . The square is
*excisive* when that comparison is a quasi-isomorphism. In that case the
homology sequence of the cover is the Mayer-Vietoris sequence of  $X$ .

The stereographic cover of  $S^n$  (complements of the poles) is the
classical excisive pair: each chart is contractible. Small simplices /
barycentric subdivision, which would prove this pair excisive, are not
in Mathlib and are not completed here; the remaining obstruction to
 $H_n(\text{Metric.sphere})$  for  $n > 0$  is exactly that quasi-isomorphism.
-/

open AlgebraicTopology CategoryTheory Limits HomologicalComplex Metric Set
open scoped Topology ContinuousMap

set_option linter.unusedSectionVars false

universe v u

variable {C : Type u} [Category.{v} C] [Abelian C]
[HasCoproducts.{0} C] [HasPullbacks C]

variable (R : C)

/-! ## Abstract triad -/

variable {W U V X : TopCat.{0}}
variable (iU : W → U) (iV : W → V) (jU : U → X) (jV : V → X)

/--  $C_*(W) \rightarrow C_*(U) \oplus C_*(V)$ , the Mayer-Vietoris inclusion of the intersection. -/
noncomputable def mvIncl :
  ((singularChainComplexFunctor C).obj R).obj W →
    ((singularChainComplexFunctor C).obj R).obj U ⊕
      ((singularChainComplexFunctor C).obj R).obj V :=

```

```

biprod.lift
  (((singularChainComplexFunctor C).obj R).map iU)
  (-(((singularChainComplexFunctor C).obj R).map iV))

instance mvIncl_mono [Mono iU] : Mono (mvIncl (C := C) R iU iV) :=
  haveI : Mono (((singularChainComplexFunctor C).obj R).map iU) :=
    (((singularChainComplexFunctor C).obj R).map_mono iU)
  mono_of_mono_fac (biprod.lift_fst
    (((singularChainComplexFunctor C).obj R).map iU)
    (-(((singularChainComplexFunctor C).obj R).map iV)))

/-- Chains of the cover: `coker(C_*(W) → C_*(U) ⊕ C_*(V))`. -/
noncomputable def coverChains : ChainComplex C ℕ :=
  cokernel (mvIncl (C := C) R iU iV)

/-- The short complex of the triad. -/
noncomputable def triadShortComplex [Mono iU] :
  ShortComplex (ChainComplex C ℕ) :=
  ShortComplex.mk (mvIncl (C := C) R iU iV) (cokernel.π _) (cokernel.condition _)

instance triadShortComplex_mono_f [Mono iU] :
  Mono (triadShortComplex (C := C) R iU iV).f :=
  inferInstanceAs (Mono (mvIncl (C := C) R iU iV))

instance triadShortComplex_epi_g [Mono iU] :
  Epi (triadShortComplex (C := C) R iU iV).g :=
  inferInstanceAs (Epi (cokernel.π (mvIncl (C := C) R iU iV)))

/-- The triad sequence of singular chains is short exact. -/
lemma triad_shortExact [Mono iU] :
  (triadShortComplex (C := C) R iU iV).ShortExact :=
  ShortComplex.ShortExact.mk'
    (ShortComplex.exact_of_g_is_cokernel _
      (cokernelIsCokernel (mvIncl (C := C) R iU iV)))
  inferInstance inferInstance

/-- `C_*(U) ⊕ C_*(V) → C_*(X)` induced by the two inclusions into `X`. -/
noncomputable def coverToX (hcomm : iU ≫ jU = iV ≫ jV) :
  (((singularChainComplexFunctor C).obj R).obj U ⊕
    (((singularChainComplexFunctor C).obj R).obj V →
      ((singularChainComplexFunctor C).obj R).obj X :=
    biprod.desc
      (((singularChainComplexFunctor C).obj R).map jU)
      (((singularChainComplexFunctor C).obj R).map jV))

lemma mvIncl_comp_coverToX (hcomm : iU ≫ jU = iV ≫ jV) :
  mvIncl (C := C) R iU iV ≫ coverToX (C := C) R iU iV jU jV hcomm = 0 := by
  dsimp [mvIncl, coverToX]
  ext
  simp [← Functor.map_comp, hcomm]

/-- Comparison of cover chains with ambient chains. -/
noncomputable def coverComparison (hcomm : iU ≫ jU = iV ≫ jV) :
  coverChains (C := C) R iU iV →
    (((singularChainComplexFunctor C).obj R).obj X :=
  cokernel.desc _ (coverToX (C := C) R iU iV jU jV hcomm)
  (mvIncl_comp_coverToX (C := C) R iU iV jU jV hcomm))

/-- A commuting square of spaces is excisive when cover chains compute `H_*(X)`. -/
def IsExcisive (hcomm : iU ≫ jU = iV ≫ jV) : Prop :=
  QuasiIso (coverComparison (C := C) R iU iV jU jV hcomm)

/-- Connecting map of the triad: `H_{n+1}(U+V) → H_n(W)`. -/

```

```

noncomputable def triad $\delta$  [Mono iU] (n :  $\mathbb{N}$ ) :
  (coverChains (C := C) R iU iV).homology (n + 1)  $\longrightarrow$ 
  (((singularChainComplexFunctor C).obj R).obj W).homology n :=
  (triad_shortExact (C := C) R iU iV). $\delta$  (n + 1) n (by simp)

/-- If the square is excisive, homology of the cover is homology of `X`. -/
noncomputable def excisiveHomologyIso
  (hcomm : iU  $\gg$  jU = iV  $\gg$  jV)
  (h : IsExcisive (C := C) R iU iV jU jV hcomm) (n :  $\mathbb{N}$ ) :
  (coverChains (C := C) R iU iV).homology n  $\cong$ 
  (((singularChainComplexFunctor C).obj R).obj X).homology n :=
  have : QuasiIso (coverComparison (C := C) R iU iV jU jV hcomm) := h
  asIso (homologyMap (coverComparison (C := C) R iU iV jU jV hcomm) n)

/-- Mayer-Vietoris connecting map, after excision. -/
noncomputable def mayerVietoris $\delta$  [Mono iU]
  (hcomm : iU  $\gg$  jU = iV  $\gg$  jV)
  (h : IsExcisive (C := C) R iU iV jU jV hcomm) (n :  $\mathbb{N}$ ) :
  (((singularChainComplexFunctor C).obj R).obj X).homology (n + 1)  $\longrightarrow$ 
  (((singularChainComplexFunctor C).obj R).obj W).homology n :=
  (excisiveHomologyIso (C := C) R iU iV jU jV hcomm h (n + 1)).inv  $\gg$ 
  triad $\delta$  (C := C) R iU iV n

/-- Exactness of ` $H_n(W) \rightarrow H_n(U) \oplus H_n(V) \rightarrow H_n(U+V)$ `. -/
lemma triad_exact_at_sum [Mono iU] (n :  $\mathbb{N}$ ) :
  (ShortComplex.mk
    (homologyMap (mvIncl (C := C) R iU iV) n)
    (homologyMap (cokernel. $\pi$  (mvIncl (C := C) R iU iV)) n)
    (by rw [← homologyMap_comp, cokernel.condition, homologyMap_zero])).Exact :=
  (triad_shortExact (C := C) R iU iV).homology_exact2 n

/-- Exactness of ` $H_{n+1}(U+V) \rightarrow H_n(W) \rightarrow H_n(U) \oplus H_n(V)$ `. -/
lemma triad_exact_at_W [Mono iU] (n :  $\mathbb{N}$ ) :
  (ShortComplex.mk (triad $\delta$  (C := C) R iU iV n)
    (homologyMap (mvIncl (C := C) R iU iV) n)
    (by
      simpa [triad $\delta$ , triadShortComplex] using
      (triad_shortExact (C := C) R iU iV). $\delta$ _comp (n + 1) n (by simp))).Exact :=
  (triad_shortExact (C := C) R iU iV).homology_exact1 (n + 1) n (by simp)

/-! ## Subspace covers -/

variable {Y : Type} [TopologicalSpace Y]

/-- Inclusion of an intersection as a subspace of the first set. -/
def interInclLeft (A B : Set Y) : TopCat.of  $\uparrow\uparrow(A \cap B) \longrightarrow$  TopCat.of  $\uparrow\uparrow A :=$ 
  TopCat.ofHom (ContinuousMap.inclusion (fun _ hx => hx.1))

/-- Inclusion of an intersection as a subspace of the second set. -/
def interInclRight (A B : Set Y) : TopCat.of  $\uparrow\uparrow(A \cap B) \longrightarrow$  TopCat.of  $\uparrow\uparrow B :=$ 
  TopCat.ofHom (ContinuousMap.inclusion (fun _ hx => hx.2))

instance interInclLeft_mono (A B : Set Y) : Mono (interInclLeft A B) :=
  (TopCat.mono_iff_injective _).2 fun _ _ h =>
  Subtype.ext (congrArg (Subtype.val :  $\uparrow\uparrow A \rightarrow Y$ ) h)

instance interInclRight_mono (A B : Set Y) : Mono (interInclRight A B) :=
  (TopCat.mono_iff_injective _).2 fun _ _ h =>
  Subtype.ext (congrArg (Subtype.val :  $\uparrow\uparrow B \rightarrow Y$ ) h)

lemma subset_square_comm (A B : Set Y) :
  interInclLeft A B  $\gg$  subtypeIncl Y A =

```

```

    interInclRight A B >> subtypeIncl Y B := by
  ext x
  rfl

/-- A pair of subspaces is excisive when their cover computes  $H_*(Y)$ . -/
def IsExcisiveSubspaces (A B : Set Y) : Prop :=
  IsExcisive (C := C) R (interInclLeft A B) (interInclRight A B)
  (subtypeIncl Y A) (subtypeIncl Y B) (subset_square_comm A B)

/-! ## Stereographic cover of the metric sphere -/

/-- The unit  $n$ -sphere in Euclidean space of dimension  $n+1$ . -/
abbrev MetricSphere (n : ℕ) : Type :=
  ↑(sphere (0 : EuclideanSpace ℝ (Fin (n + 1))) 1)

/-- North pole  $e_0$  of  $S^n$ . -/
noncomputable def northPole (n : ℕ) : MetricSphere n :=
  ⟨EuclideanSpace.single 0 1, by simp [mem_sphere_iff_norm]⟩

/-- South pole  $-e_0$  of  $S^n$ . -/
noncomputable def southPole (n : ℕ) : MetricSphere n :=
  ⟨-EuclideanSpace.single 0 1, by simp [mem_sphere_iff_norm]⟩

lemma northPole_ne_southPole (n : ℕ) : northPole n ≠ southPole n := by
  intro h
  have h1 : (1 : ℝ) = -1 := by
    simp [northPole, southPole] using
      congrArg (fun p : MetricSphere n =>
        (p : EuclideanSpace ℝ (Fin (n + 1))) 0) h
  norm_num at h1

/-- Open chart  $S^n \setminus \{N\}$ . -/
def puncturedNorth (n : ℕ) : Set (MetricSphere n) :=
  {northPole n}c

/-- Open chart  $S^n \setminus \{S\}$ . -/
def puncturedSouth (n : ℕ) : Set (MetricSphere n) :=
  {southPole n}c

lemma isOpen_puncturedNorth (n : ℕ) : IsOpen (puncturedNorth n) :=
  isOpen_compl_singleton

lemma isOpen_puncturedSouth (n : ℕ) : IsOpen (puncturedSouth n) :=
  isOpen_compl_singleton

lemma punctured_cover (n : ℕ) :
  puncturedNorth n ∪ puncturedSouth n = univ := by
  ext x
  simp [puncturedNorth, puncturedSouth]
  exact or_iff_not_imp_left.2 fun hx => by
    rintro rfl
    exact hx (northPole_ne_southPole n).symm

lemma punctured_inter (n : ℕ) :
  puncturedNorth n ∩ puncturedSouth n = {northPole n, southPole n}c := by
  ext x
  simp [puncturedNorth, puncturedSouth]

lemma norm_northPole (n : ℕ) :
  ||(northPole n : EuclideanSpace ℝ (Fin (n + 1)))|| = 1 := by
  simp [northPole]

lemma norm_southPole (n : ℕ) :

```

```

    ||(southPole n : EuclideanSpace ℝ (Fin (n + 1)))|| = 1 := by
simp [southPole]

/-- Stereographic projection from the north pole is a homeomorphism
`S^n \ {N}` homeomorphic to the orthogonal complement of `ℝ · N`. -/
noncomputable def homeoPuncturedNorth (n : ℕ) :
  ↑(puncturedNorth n) ≃ₜ
    ((ℝ · (northPole n : EuclideanSpace ℝ (Fin (n + 1))))⊥) :=
have hsrc : (stereographic (norm_northPole n)).source = puncturedNorth n := by
ext x
simp [puncturedNorth, northPole, stereographic_source]
(Homeomorph.setCongr hsrc).trans
((stereographic (norm_northPole n)).toHomeomorphSourceTarget.trans
(Homeomorph.Set.univ _))

instance contractible_puncturedNorth (n : ℕ) :
  ContractibleSpace ↑(puncturedNorth n) :=
(homeoPuncturedNorth n).contractibleSpace

/-- Stereographic projection from the south pole is a homeomorphism
`S^n \ {S}` homeomorphic to the orthogonal complement of `ℝ · S`. -/
noncomputable def homeoPuncturedSouth (n : ℕ) :
  ↑(puncturedSouth n) ≃ₜ
    ((ℝ · (southPole n : EuclideanSpace ℝ (Fin (n + 1))))⊥) :=
have hsrc : (stereographic (norm_southPole n)).source = puncturedSouth n := by
ext x
simp [puncturedSouth, southPole, stereographic_source]
(Homeomorph.setCongr hsrc).trans
((stereographic (norm_southPole n)).toHomeomorphSourceTarget.trans
(Homeomorph.Set.univ _))

instance contractible_puncturedSouth (n : ℕ) :
  ContractibleSpace ↑(puncturedSouth n) :=
(homeoPuncturedSouth n).contractibleSpace

/-- Positive singular homology of the northern stereographic chart vanishes. -/
lemma isZero_singularHomology_puncturedNorth
  (n : ℕ) {k : ℕ} (hk : k ≠ 0) :
  IsZero (((singularHomologyFunctor C k).obj R).obj
    (.of ↑(puncturedNorth n))) :=
isZero_singularHomology_of_contractible R hk

/-- Positive singular homology of the southern stereographic chart vanishes. -/
lemma isZero_singularHomology_puncturedSouth
  (n : ℕ) {k : ℕ} (hk : k ≠ 0) :
  IsZero (((singularHomologyFunctor C k).obj R).obj
    (.of ↑(puncturedSouth n))) :=
isZero_singularHomology_of_contractible R hk

/-- Mayer-Vietoris connecting map for the stereographic cover, after
excision. The charts are contractible, so this is the inductive step
for `H_n(S^n)` once `IsExcisiveSubspaces` is available. -/
noncomputable def singularHomology_sphere_mayerVietorisδ
  (n : ℕ)
  (hex : IsExcisiveSubspaces (C := C) R
    (puncturedNorth (n + 1)) (puncturedSouth (n + 1))) :
  (((singularChainComplexFunctor C).obj R).obj
    (.of (MetricSphere (n + 1)))) .homology (n + 1) →
    (((singularChainComplexFunctor C).obj R).obj
    (.of ↑(puncturedNorth (n + 1) ∩ puncturedSouth (n + 1)))).homology n :=
mayerVietorisδ (C := C) R
  (interInclLeft (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))
  (interInclRight (puncturedNorth (n + 1)) (puncturedSouth (n + 1)))

```

```

(subtypeIncl (MetricSphere (n + 1)) (puncturedNorth (n + 1)))
(subtypeIncl (MetricSphere (n + 1)) (puncturedSouth (n + 1)))
(subset_square_comm _ _) hex n

```

A.7 PlatonicSolids/Sphere2Homology.lean

Cellular $H_*(S^2; \mathbb{Q})$ of the tetrahedron surface (129 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.Matrix.Notation
import Mathlib.LinearAlgebra.Matrix.Rank
import Mathlib.LinearAlgebra.Matrix.ToLin
import Mathlib.Tactic
import PlatonicSolids.EulerPoincare
import PlatonicSolids.Incidence

/-!
Paper §2.1. Cellular homology of the standard simplicial  $(S^2)$ :
the boundary of a tetrahedron (4 vertices, 6 edges, 4 triangles).

This is the usual CW / simplicial calculation
 $H_0 \cong \mathbb{Q}$ ,  $H_1 = 0$ ,  $H_2 \cong \mathbb{Q}$ . Singular  $H_0$  of Metric.sphere
and  $H_*(S^0)$  are in PlatonicSolids/SingularHomology.lean;
the  $S^3$ -sphere calculation is PlatonicSolids/Sphere3Homology.lean.
-/

open scoped Matrix
open Matrix Module

/-- Oriented incidence vertices  $\leftarrow$  edges: columns 01,02,03,12,13,23. -/
def tetra_d1 : Matrix (Fin 4) (Fin 6)  $\mathbb{Q}$  :=
  !![ -1, -1, -1, 0, 0, 0;
      1, 0, 0, -1, -1, 0;
      0, 1, 0, 1, 0, -1;
      0, 0, 1, 0, 1, 1]

/-- Oriented incidence edges  $\leftarrow$  faces: columns 012,013,023,123. -/
def tetra_d2 : Matrix (Fin 6) (Fin 4)  $\mathbb{Q}$  :=
  !![ 1, 1, 0, 0;
      -1, 0, 1, 0;
      0, -1, -1, 0;
      1, 0, 0, 1;
      0, 1, 0, -1;
      0, 0, 1, 1]

lemma tetra_comp : tetra_d1 * tetra_d2 = 0 := by
  ext i j
  fin_cases i <;> fin_cases j <;> simp [tetra_d1, tetra_d2]

noncomputable def tetra_del1 : (Fin 6  $\rightarrow$   $\mathbb{Q}$ )  $\rightarrow_1$  [ $\mathbb{Q}$ ] (Fin 4  $\rightarrow$   $\mathbb{Q}$ ) := tetra_d1.toLin'
noncomputable def tetra_del2 : (Fin 4  $\rightarrow$   $\mathbb{Q}$ )  $\rightarrow_1$  [ $\mathbb{Q}$ ] (Fin 6  $\rightarrow$   $\mathbb{Q}$ ) := tetra_d2.toLin'

lemma tetra_del_comp : tetra_del1.comp tetra_del2 = 0 := by
  rw [tetra_del1, tetra_del2,  $\leftarrow$  toLin'_mul, tetra_comp]
  exact map_zero _

lemma tetra_d1_rank_ge :  $3 \leq$  tetra_d1.rank := by
  have hI : ((1 : Matrix (Fin 3) (Fin 3)  $\mathbb{Q}$ ).rank) = 3 := rank_one
  have hsub :
    tetra_d1.submatrix ![1, 2, 3] ![0, 1, 2] = (1 : Matrix (Fin 3) (Fin 3)  $\mathbb{Q}$ ) := by

```

```

    ext i j
    fin_cases i <;> fin_cases j <;> simp [tetra_d1]
have : (tetra_d1.submatrix ![1, 2, 3] ![0, 1, 2]).rank = 3 := by
  rw [hsub]; exact hI
exact this.symm.trans_le (rank_submatrix_le _ _ _)

lemma tetra_d2_rank_ge : 3 ≤ tetra_d2.rank := by
  have hI : ((1 : Matrix (Fin 3) (Fin 3) ℚ).rank) = 3 := rank_one
  have hsub :
    tetra_d2.submatrix ![3, 4, 5] ![0, 1, 2] = (1 : Matrix (Fin 3) (Fin 3) ℚ) := by
    ext i j
    fin_cases i <;> fin_cases j <;> simp [tetra_d2]
  have : (tetra_d2.submatrix ![3, 4, 5] ![0, 1, 2]).rank = 3 := by
    rw [hsub]; exact hI
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

lemma tetra_d1_rank : tetra_d1.rank = 3 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero tetra_comp
  have h1 := tetra_d1_rank_ge
  have h2 := tetra_d2_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma tetra_d2_rank : tetra_d2.rank = 3 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero tetra_comp
  have h1 := tetra_d1_rank_ge
  have h2 := tetra_d2_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma tetra_del1_range : finrank ℚ (LinearMap.range tetra_del1) = 3 := by
  simpa [tetra_del1, toLin'_apply'] using tetra_d1_rank

lemma tetra_del2_range : finrank ℚ (LinearMap.range tetra_del2) = 3 := by
  simpa [tetra_del2, toLin'_apply'] using tetra_d2_rank

/--  $H_0(S^2; \mathbb{Q}) \cong \mathbb{Q}$ . -/
theorem sphere2_betti0 : betti0 tetra_del1 = 1 := by
  simp [betti0, tetra_del1_range, finrank_fintype_fun_eq_card]

/--  $H_1(S^2; \mathbb{Q}) = 0$ . -/
theorem sphere2_betti1 : betti1 tetra_del2 tetra_del1 = 0 := by
  have hr := tetra_del1.finrank_range_add_finrank_ker
  simp [betti1, tetra_del2_range, tetra_del1_range, finrank_fintype_fun_eq_card] at hr ⊢
  omega

/--  $H_2(S^2; \mathbb{Q}) \cong \mathbb{Q}$ . -/
theorem sphere2_betti2 : betti2 tetra_del2 = 1 := by
  have hr := tetra_del2.finrank_range_add_finrank_ker
  simp [betti2, tetra_del2_range, finrank_fintype_fun_eq_card] at hr ⊢
  omega

/-- Cellular homology of the standard simplicial 2-sphere. -/
theorem homology_sphere2 :
  betti0 tetra_del1 = 1 ∧ betti1 tetra_del2 tetra_del1 = 0 ∧ betti2 tetra_del2 = 1 :=
  ⟨sphere2_betti0, sphere2_betti1, sphere2_betti2⟩

/-- Euler's formula on the tetrahedron, from its computed Betti numbers. -/
theorem euler_tetrahedron : (4 : ℕ) + 4 = 6 + 2 :=
  euler_formula_of_sphere2 tetra_del2 tetra_del1
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])

```

```

tetra_del_comp sphere2_betti0 sphere2_betti1 sphere2_betti2

/-- The tetrahedron incidence structure is Platonic, using homology not a
raw Euler hypothesis. -/
theorem tetrahedron_platonic : IsPlatonicPair 3 3 :=
  platonic_solids_3d_of_homology tetra_del2 tetra_del1 4 6 4 3 3
  (by decide) (by decide) (by decide) (by decide)
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  tetra_del_comp sphere2_betti0 sphere2_betti1 sphere2_betti2

```

A.8 PlatonicSolids/Sphere3Homology.lean

Cellular $H_*(S^3; \mathbb{Q})$ of the 4-simplex boundary (195 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.Matrix.Notation
import Mathlib.LinearAlgebra.Matrix.Rank
import Mathlib.LinearAlgebra.Matrix.ToLin
import Mathlib.Tactic
import PlatonicSolids.EulerPoincare

/-!
Paper §3.1. Cellular homology of the standard simplicial  $(S^3)$ :
the boundary of a 4-simplex (5 vertices, 10 edges, 10 triangles,
5 tetrahedra).

Ranks are  $\{4, 6, 4\}$ , so  $b_0 = 1$ ,  $b_1 = 0$ ,  $b_2 = 0$ ,  $b_3 = 1$  and
 $\chi = 0$ . This is the combinatorial 3-sphere the paper uses; Mathlib
does not compute singular homology of Metric.sphere in positive
degree.
-/

open scoped Matrix
open Matrix Module

/-- Vertices  $\{0..4\} \leftarrow$  edges  $\{01,02,03,04,12,13,14,23,24,34\}$ . -/
def pentatope_d1 : Matrix (Fin 5) (Fin 10)  $\mathbb{Q}$  :=
  !![ -1, -1, -1, -1, 0, 0, 0, 0, 0, 0;
      1, 0, 0, 0, -1, -1, -1, 0, 0, 0;
      0, 1, 0, 0, 1, 0, 0, -1, -1, 0;
      0, 0, 1, 0, 0, 1, 0, 1, 0, -1;
      0, 0, 0, 1, 0, 0, 1, 0, 1, 1]

/-- Edges  $\leftarrow$  triangles
 $\{012,013,014,023,024,034,123,124,134,234\}$ . -/
def pentatope_d2 : Matrix (Fin 10) (Fin 10)  $\mathbb{Q}$  :=
  !![ 1, 1, 1, 0, 0, 0, 0, 0, 0, 0;
      -1, 0, 0, 1, 1, 0, 0, 0, 0, 0;
      0, -1, 0, -1, 0, 1, 0, 0, 0, 0;
      0, 0, -1, 0, -1, -1, 0, 0, 0, 0;
      1, 0, 0, 0, 0, 0, 1, 1, 0, 0;
      0, 1, 0, 0, 0, 0, -1, 0, 1, 0;
      0, 0, 1, 0, 0, 0, 0, -1, -1, 0;
      0, 0, 0, 1, 0, 0, 1, 0, 0, 1;
      0, 0, 0, 0, 1, 0, 0, 1, 0, -1;
      0, 0, 0, 0, 0, 1, 0, 0, 1, 1]

/-- Triangles  $\leftarrow$  tetrahedra  $\{0123,0124,0134,0234,1234\}$ . -/

```

```

def pentatope_d3 : Matrix (Fin 10) (Fin 5) ℚ :=
  !![ -1, -1, 0, 0, 0;
      1, 0, -1, 0, 0;
      0, 1, 1, 0, 0;
      -1, 0, 0, -1, 0;
      0, -1, 0, 1, 0;
      0, 0, -1, -1, 0;
      1, 0, 0, 0, -1;
      0, 1, 0, 0, 1;
      0, 0, 1, 0, -1;
      0, 0, 0, 1, 1]

lemma pentatope_comp21 : pentatope_d1 * pentatope_d2 = 0 := by
  native_decide

lemma pentatope_comp32 : pentatope_d2 * pentatope_d3 = 0 := by
  native_decide

noncomputable def pentatope_del1 : (Fin 10 → ℚ) →1[ℚ] (Fin 5 → ℚ) :=
  pentatope_d1.toLin'
noncomputable def pentatope_del2 : (Fin 10 → ℚ) →1[ℚ] (Fin 10 → ℚ) :=
  pentatope_d2.toLin'
noncomputable def pentatope_del3 : (Fin 5 → ℚ) →1[ℚ] (Fin 10 → ℚ) :=
  pentatope_d3.toLin'

lemma pentatope_del_comp21 : pentatope_del1.comp pentatope_del2 = 0 := by
  rw [pentatope_del1, pentatope_del2, ← toLin'_mul, pentatope_comp21]
  exact map_zero _

lemma pentatope_del_comp32 : pentatope_del2.comp pentatope_del3 = 0 := by
  rw [pentatope_del2, pentatope_del3, ← toLin'_mul, pentatope_comp32]
  exact map_zero _

lemma pentatope_d1_rank_ge : 4 ≤ pentatope_d1.rank := by
  have hI : ((1 : Matrix (Fin 4) (Fin 4) ℚ).rank) = 4 := rank_one
  have hsub :
    pentatope_d1.submatrix ![1, 2, 3, 4] ![0, 1, 2, 3] =
      (1 : Matrix (Fin 4) (Fin 4) ℚ) := by
    ext i j
    fin_cases i <> fin_cases j <> simp [pentatope_d1]
  have : (pentatope_d1.submatrix ![1, 2, 3, 4] ![0, 1, 2, 3]).rank = 4 := by
    rw [hsub]; exact hI
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

lemma pentatope_d3_rank_ge : 4 ≤ pentatope_d3.rank := by
  have hI : ((1 : Matrix (Fin 4) (Fin 4) ℚ).rank) = 4 := rank_one
  have hsub :
    pentatope_d3.submatrix ![6, 7, 8, 9] ![0, 1, 2, 3] =
      (1 : Matrix (Fin 4) (Fin 4) ℚ) := by
    ext i j
    fin_cases i <> fin_cases j <> simp [pentatope_d3]
  have : (pentatope_d3.submatrix ![6, 7, 8, 9] ![0, 1, 2, 3]).rank = 4 := by
    rw [hsub]; exact hI
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

/-- A `6 × 6` minor of `d2` with determinant `1`. -/
def pentatope_d2_minor : Matrix (Fin 6) (Fin 6) ℚ :=
  !![ 1, 1, 1, 0, 0, 0;
      -1, 0, 0, 1, 1, 0;
      0, -1, 0, -1, 0, 1;
      1, 0, 0, 0, 0, 0;
      0, 1, 0, 0, 0, 0;
      0, 0, 0, 1, 0, 0]

```

```

lemma pentatope_d2_minor_eq :
  pentatope_d2.submatrix ![0, 1, 2, 4, 5, 7] ![0, 1, 2, 3, 4, 5] =
    pentatope_d2_minor := by
  ext i j
  fin_cases i <;> fin_cases j <;> simp [pentatope_d2, pentatope_d2_minor]

lemma pentatope_d2_minor_det : pentatope_d2_minor.det = 1 := by
  native_decide

lemma pentatope_d2_rank_ge : 6 ≤ pentatope_d2.rank := by
  have hU : IsUnit pentatope_d2_minor :=
    (Matrix.isUnit_iff_isUnit_det _).2 (by simp [pentatope_d2_minor_det])
  have : pentatope_d2_minor.rank = 6 := by
    simp using rank_of_isUnit pentatope_d2_minor hU
  rw [← pentatope_d2_minor_eq] at this
  exact this.symm.trans_le (rank_submatrix_le _ _ _)

lemma pentatope_d1_rank : pentatope_d1.rank = 4 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero pentatope_comp21
  have h1 := pentatope_d1_rank_ge
  have h2 := pentatope_d2_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma pentatope_d2_rank : pentatope_d2.rank = 6 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero pentatope_comp21
  have h1 := pentatope_d1_rank_ge
  have h2 := pentatope_d2_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma pentatope_d3_rank : pentatope_d3.rank = 4 := by
  have hsum := rank_add_rank_le_card_of_mul_eq_zero pentatope_comp32
  have h2 := pentatope_d2_rank
  have h3 := pentatope_d3_rank_ge
  simp only [Fintype.card_fin] at hsum
  omega

lemma pentatope_del1_range : finrank ℚ (LinearMap.range pentatope_del1) = 4 := by
  simp [pentatope_del1, toLin'.apply'] using pentatope_d1_rank

lemma pentatope_del2_range : finrank ℚ (LinearMap.range pentatope_del2) = 6 := by
  simp [pentatope_del2, toLin'.apply'] using pentatope_d2_rank

lemma pentatope_del3_range : finrank ℚ (LinearMap.range pentatope_del3) = 4 := by
  simp [pentatope_del3, toLin'.apply'] using pentatope_d3_rank

theorem sphere3_betti0 : betti0 pentatope_del1 = 1 := by
  simp [betti0, pentatope_del1_range, finrank_fintype_fun_eq_card]

theorem sphere3_betti1 : betti1 pentatope_del2 pentatope_del1 = 0 := by
  have hr := pentatope_del1.finrank_range_add_finrank_ker
  simp [betti1, pentatope_del2_range, pentatope_del1_range,
    finrank_fintype_fun_eq_card] at hr ⊢
  omega

theorem sphere3_betti2 : betti2Rel pentatope_del3 pentatope_del2 = 0 := by
  have hr := pentatope_del2.finrank_range_add_finrank_ker
  simp [betti2Rel, pentatope_del3_range, pentatope_del2_range,
    finrank_fintype_fun_eq_card] at hr ⊢
  omega

```

```

theorem sphere3_betti3 : betti3 pentatope_del3 = 1 := by
  have hr := pentatope_del3.finrank_range_add_finrank_ker
  simp [betti3, pentatope_del3_range, finrank_fintype_fun_eq_card] at hr ⊢
  omega

/-- Cellular homology of the standard simplicial 3-sphere. -/
theorem homology_sphere3 :
  betti0 pentatope_del1 = 1 ∧
  betti1 pentatope_del2 pentatope_del1 = 0 ∧
  betti2Rel pentatope_del3 pentatope_del2 = 0 ∧
  betti3 pentatope_del3 = 1 :=
  ⟨sphere3_betti0, sphere3_betti1, sphere3_betti2, sphere3_betti3⟩

/--  $\chi(S^3) = 0$  from the computed Betti numbers of the 4-simplex boundary. -/
theorem eulerChar_sphere3 : ((5 : ℕ) : ℤ) - 10 + 10 - 5 = 0 :=
  euler_formula_of_sphere3 pentatope_del3 pentatope_del2 pentatope_del1
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  (by simp [finrank_fintype_fun_eq_card])
  pentatope_del_comp32 pentatope_del_comp21
  sphere3_betti0 sphere3_betti1 sphere3_betti2 sphere3_betti3

```

A.9 PlatonicSolids/EulerBetti.lean

Betti arithmetic of $\chi(S^2) = 2$ and $\chi(S^3) = 0$ (24 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Tactic

/-!
Paper §2.1 / §3.1. Betti arithmetic of  $\chi(S^2)$  and  $\chi(S^3)$ .

The algebraic Euler-Poincaré identities are `euler_poincare` and
`euler_poincare3` in `PlatonicSolids/EulerPoincare.lean`. The numbers
`b_0 = 1`, `b_1 = 0`, `b_2 = 1` for the standard simplicial `S^2` are
`homology_sphere2`; the numbers `b_0 = 1`, `b_1 = 0`, `b_2 = 0`,
`b_3 = 1` for the standard simplicial `S^3` are `homology_sphere3`.
Singular `H_0` of `Metric.sphere` in dimension `n ≥ 1` and the full
calculation of `H_*(S^0)` are in `PlatonicSolids/SingularHomology.lean`.
These lemmas record the resulting  $\chi(S^2) = 2$  and  $\chi(S^3) = 0$ .
-/

/--  $\chi(S^2) = b_0 - b_1 + b_2 = 1 - 0 + 1$ . -/
lemma eulerChar_sphere2_betti : (1 : ℤ) - 0 + 1 = 2 := by norm_num

/--  $\chi(S^3) = b_0 - b_1 + b_2 - b_3 = 1 - 0 + 0 - 1$ . -/
lemma eulerChar_sphere3_betti : (1 : ℤ) - 0 + 0 - 1 = 0 := by norm_num

```

A.10 PlatonicSolids/PlatonicPair.lean

The five solutions of the Platonic Diophantine inequality (38 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Tactic

```

```

/-!
Paper §2.3. The five solutions of  $(p-2)(q-2) < 4$  for  $p, q \geq 3$ .
-/

/-- Schlöfli pairs of the five Platonic solids. -/
def IsPlatonicPair (p q : ℕ) : Prop :=
  (p = 3 ∧ q = 3) ∨
  (p = 3 ∧ q = 4) ∨
  (p = 4 ∧ q = 3) ∨
  (p = 3 ∧ q = 5) ∨
  (p = 5 ∧ q = 3)

lemma pq_ge_two_p_add_two_q {p q : ℕ} (hp : 4 ≤ p) (hq : 4 ≤ q) :
  2 * p + 2 * q ≤ p * q := by
  zify
  nlinarith

/-- The inequality  $pq < 2p + 2q$  for  $p, q \geq 3$  has exactly the five Platonic solutions. -/
theorem platonic_pairs_of_inequality {p q : ℕ}
  (hp : 3 ≤ p) (hq : 3 ≤ q) (h : p * q < 2 * p + 2 * q) :
  IsPlatonicPair p q := by
  rcases show p = 3 ∨ q = 3 ∨ (4 ≤ p ∧ 4 ≤ q) by omega with rfl | rfl | ⟨hp4, hq4⟩
  · rcases show q = 3 ∨ q = 4 ∨ q = 5 by omega with rfl | rfl | rfl <=> simp [IsPlatonicPair]
  · rcases show p = 3 ∨ p = 4 ∨ p = 5 by omega with rfl | rfl | rfl <=> simp [IsPlatonicPair]
  · exact (lt_irrefl _ (h.trans_le (pq_ge_two_p_add_two_q hp4 hq4))).elim

theorem IsPlatonicPair.ge_three {p q : ℕ} (h : IsPlatonicPair p q) : 3 ≤ p ∧ 3 ≤ q := by
  rcases h with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ <=> decide

theorem IsPlatonicPair.inequality {p q : ℕ} (h : IsPlatonicPair p q) :
  p * q < 2 * p + 2 * q := by
  rcases h with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ <=> decide

```

A.11 PlatonicSolids/Reciprocal.lean

$1/p + 1/q > 1/2$ iff the integer inequality (44 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Data.Real.Basic
import PlatonicSolids.PlatonicPair

/-!
Paper §2.3 / §3.3.  $1/p + 1/q > 1/2$  is the reciprocal form of the
Platonic Diophantine inequality, and is the condition  $D_3 > 0$ .
-/

lemma reciprocal_iff {p q : ℕ} (hp : 0 < p) (hq : 0 < q) :
  (1 / (p : ℝ) + 1 / q > 1 / 2) ↔ p * q < 2 * p + 2 * q := by
  have : (0 : ℝ) < p := Nat.cast_pos.mpr hp
  have : (0 : ℝ) < q := Nat.cast_pos.mpr hq
  constructor
  · intro h
    have h' : (p + q : ℝ) / (p * q) > 1 / 2 := by
      convert h using 1
      field_simp
      ring
    rw [gt_iff_lt, lt_div_iff0 (by positivity)] at h'
    exact_mod_cast (by linarith : (p * q : ℝ) < 2 * p + 2 * q)
  · intro h
    have hR : (p * q : ℝ) < 2 * p + 2 * q := mod_cast h

```

```

have h' : (p + q : ℝ) / (p * q) > 1 / 2 := by
  rw [gt_iff_lt, lt_div_iff0 (by positivity)]
  linarith
convert h' using 1
field_simp
ring

theorem platonic_pair_iff {p q : ℕ} :
  IsPlatonicPair p q ↔ 3 ≤ p ∧ 3 ≤ q ∧ p * q < 2 * p + 2 * q := by
  constructor
  · intro h
    exact ⟨h.ge_three.1, h.ge_three.2, h.inequality⟩
  · exact fun ⟨hp, hq, h⟩ => platonic_pairs_of_inequality hp hq h

theorem platonic_pair_iff_reciprocal {p q : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) :
  IsPlatonicPair p q ↔ 1 / (p : ℝ) + 1 / q > 1 / 2 := by
  rw [platonic_pair_iff, reciprocal_iff (by omega) (by omega)]
  simp [hp, hq]

```

A.12 PlatonicSolids/Incidence.lean

Regularity incidence plus Euler force a Platonic pair and $E > 0$ (81 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.EulerPoincare
import PlatonicSolids.PlatonicPair

/-!
Paper §2.2-2.4. Regularity incidence `pF = 2E`, `qV = 2E` and Euler
`V + F = E + 2` force a Platonic pair, and force `E > 0` with no extra
non-degeneracy axiom.
-/

/-- The Euler + regularity identities algebraically prohibit `E = 0`. -/
theorem edges_pos_of_regular
  (V E F p q : ℕ)
  (hp : 3 ≤ p) (hq : 3 ≤ q)
  (h_edges_faces : p * F = 2 * E)
  (h_edges_verts : q * V = 2 * E)
  (h_euler : V + F = E + 2) :
  0 < E := by
  have h_lhs : p * q * (V + F) = (2 * p + 2 * q) * E := calc
    p * q * (V + F) = p * (q * V) + q * (p * F) := by ring
    _ = p * (2 * E) + q * (2 * E) := by rw [h_edges_verts, h_edges_faces]
    _ = (2 * p + 2 * q) * E := by ring
  have h_rhs : p * q * (V + F) = p * q * E + 2 * p * q := calc
    p * q * (V + F) = p * q * (E + 2) := by rw [h_euler]
    _ = p * q * E + 2 * p * q := by ring
  have h_eq : (2 * p + 2 * q) * E = p * q * E + 2 * p * q := by rw [← h_lhs, h_rhs]
  by_contra hz
  have hE0 : E = 0 := Nat.eq_zero_of_not_pos hz
  subst hE0
  simp only [mul_zero, zero_add] at h_eq
  exact (Nat.pos_iff_ne_zero.mp (by positivity : 0 < 2 * p * q)) h_eq.symm

/-- Any regular incidence structure with  $\chi = 2$  is a Platonic pair. -/
theorem platonic_solids_3d
  (V E F p q : ℕ)
  (hp : 3 ≤ p) (hq : 3 ≤ q)
  (h_edges_faces : p * F = 2 * E)

```

```

(h_edges_verts : q * V = 2 * E)
(h_euler : V + F = E + 2) :
IsPlatonicPair p q := by
have h_lhs : p * q * (V + F) = (2 * p + 2 * q) * E := calc
p * q * (V + F) = p * (q * V) + q * (p * F) := by ring
_ = p * (2 * E) + q * (2 * E) := by rw [h_edges_verts, h_edges_faces]
_ = (2 * p + 2 * q) * E := by ring
have h_eq : (2 * p + 2 * q) * E = p * q * E + 2 * p * q := by
calc (2 * p + 2 * q) * E = p * q * (V + F) := h_lhs.symm
_ = p * q * (E + 2) := by rw [h_euler]
_ = p * q * E + 2 * p * q := by ring
have h_ineq : p * q < 2 * p + 2 * q := by
obtain hlt | hge := lt_or_ge (p * q) (2 * p + 2 * q)
· exact hlt
· have : (2 * p + 2 * q) * E ≤ p * q * E := Nat.mul_le_mul_right E hge
have : 0 < 2 * p * q := by positivity
omega
exact platonic_pairs_of_inequality hp hq h_ineq

/-- The 3D classification from a cellular 2-complex with the Betti
numbers of `S^2`. Euler-Poincaré (`euler_formula_of_sphere2`) supplies
`V + F = E + 2`; regularity incidence then forces a Platonic pair. -/
theorem platonic_solids_3d_of_homology
{K : Type*} [Field K]
{C₂ C₁ C₀ : Type*}
[AddCommGroup C₂] [Module K C₂] [FiniteDimensional K C₂]
[AddCommGroup C₁] [Module K C₁] [FiniteDimensional K C₁]
[AddCommGroup C₀] [Module K C₀] [FiniteDimensional K C₀]
(d₂ : C₂ →₁[K] C₁) (d₁ : C₁ →₁[K] C₀)
(V E F p q : ℕ)
(hp : 3 ≤ p) (hq : 3 ≤ q)
(h_edges_faces : p * F = 2 * E)
(h_edges_verts : q * V = 2 * E)
(hV : Module.finrank K C₀ = V)
(hE : Module.finrank K C₁ = E)
(hF : Module.finrank K C₂ = F)
(hcomp : d₁.comp d₂ = 0)
(hb0 : betti0 d₁ = 1) (hb1 : betti1 d₂ d₁ = 0) (hb2 : betti2 d₂ = 1) :
IsPlatonicPair p q :=
platonic_solids_3d V E F p q hp hq h_edges_faces h_edges_verts
(euler_formula_of_sphere2 d₂ d₁ hV hE hF hcomp hb0 hb1 hb2)

```

A.13 PlatonicSolids/Angles.lean

$\pi/n \in (0, \pi/2)$ so sine and cosine are positive (28 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic

/-!
Paper §3.3. For `n ≥ 3`, the Schläfli angle ` $\pi/n$ ` lies in ` $(0, \pi/2)$ `,
so ` $\sin$ ` and ` $\cos$ ` are positive.
-/

```

open Real

```

lemma angle_pos {n : ℕ} (hn : 1 ≤ n) : 0 < π / n := by
have : (0 : ℝ) < n := Nat.cast_pos.mpr hn
positivity

```

```

lemma angle_lt_pi_div_two {n : ℕ} (hn : 3 ≤ n) : π / n < π / 2 :=
  div_lt_div_of_pos_left pi_pos (by positivity : (0 : ℝ) < 2)
    (by exact_mod_cast (lt_of_lt_of_le (by decide : (2 : ℕ) < 3) hn))

lemma sin_angle_pos {n : ℕ} (hn : 3 ≤ n) : 0 < sin (π / n) :=
  sin_pos_of_pos_of_lt_pi (angle_pos (Nat.succ_le_of_lt (Nat.zero_lt_of_lt hn)))
    (angle_lt_pi_div_two hn |>.trans (by linarith [pi_pos]))

lemma cos_angle_pos {n : ℕ} (hn : 3 ≤ n) : 0 < cos (π / n) :=
  cos_pos_of_mem_Ioo (by linarith [angle_pos (Nat.succ_le_of_lt (Nat.zero_lt_of_lt hn)), pi_pos
    ],
    angle_lt_pi_div_two hn)

```

A.14 PlatonicSolids/GramMatrix.lean

Schläfli Gram matrix and the scalar Δ (30 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic
import Mathlib.LinearAlgebra.Matrix.Hermitian
import Mathlib.LinearAlgebra.Matrix.Notation

/-!
Paper §3.2. The Schläfli Gram matrix of  $\{p, q, r\}$  and the scalar
 $\Delta(p, q, r)$  that will equal its determinant.
-/

open scoped Matrix
open Real Matrix

/-- Canonical bilinear form of the rank-4 Schläfli Coxeter system  $\{p, q, r\}$ . -/
noncomputable def schlaflGram (p q r : ℕ) : Matrix (Fin 4) (Fin 4) ℝ :=
  !![1, -cos (π / p), 0, 0;
    -cos (π / p), 1, -cos (π / q), 0;
    0, -cos (π / q), 1, -cos (π / r);
    0, 0, -cos (π / r), 1]

/--  $\Delta(p, q, r) = \sin^2(\pi/p) \sin^2(\pi/r) - \cos^2(\pi/q)$ . -/
noncomputable def schlaflDet (p q r : ℕ) : ℝ :=
  sin (π / p) ^ 2 * sin (π / r) ^ 2 - cos (π / q) ^ 2

lemma schlaflGram_isHermitian (p q r : ℕ) : (schlaflGram p q r).IsHermitian := by
  ext i j
  fin_cases i <;> fin_cases j <;> simp [schlaflGram, conjTranspose]

```

A.15 PlatonicSolids/LeadingMinors.lean

Closed forms of the leading minors D_1, D_2, D_3 (34 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
import PlatonicSolids.GramMatrix

/-!
Paper §3.3-3.4. Leading principal minors  $D_1$ ,  $D_2$ ,  $D_3$  of the Gram
matrix, matching the paper's closed forms.

```

```

-/

open scoped Matrix
open Real Matrix

def leadingSubmatrix (k : ℕ) (hk : k ≤ 4) (A : Matrix (Fin 4) (Fin 4) ℝ) :
  Matrix (Fin k) (Fin k) ℝ :=
  A.submatrix (Fin.castLE hk) (Fin.castLE hk)

noncomputable def leadingMinor (k : ℕ) (hk : k ≤ 4) (p q r : ℕ) : ℝ :=
  (leadingSubmatrix k hk (schlafligram p q r)).det

lemma leadingMinor_one (p q r : ℕ) : leadingMinor 1 (by decide) p q r = 1 := by
  simp [leadingMinor, leadingSubmatrix, schlafligram]

lemma leadingMinor_two (p q r : ℕ) :
  leadingMinor 2 (by decide) p q r = sin (π / p) ^ 2 := by
  simp [leadingMinor, leadingSubmatrix, schlafligram, det_fin_two]
  linarith [sin_sq_add_cos_sq (π / p)]

lemma leadingMinor_three (p q r : ℕ) :
  leadingMinor 3 (by decide) p q r = sin (π / p) ^ 2 - cos (π / q) ^ 2 := by
  simp [leadingMinor, leadingSubmatrix, schlafligram, det_fin_three]
  linarith [sin_sq_add_cos_sq (π / p)]

```

A.16 PlatonicSolids/TrailingMinor.lean

Trailing 3×3 minor is the vertex-figure cell minor (22 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/

import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
import PlatonicSolids.GramMatrix

/-!
Paper §3.3. The trailing  $3 \times 3$  principal minor is the cell minor of the
vertex figure  $\{q, r\}$ .
-/

open scoped Matrix
open Real Matrix

noncomputable def trailing3 (p q r : ℕ) : Matrix (Fin 3) (Fin 3) ℝ :=
  (schlafligram p q r).submatrix (fun i : Fin 3 => i.succ) (fun i => i.succ)

lemma trailing3_det (p q r : ℕ) :
  (trailing3 p q r).det = sin (π / q) ^ 2 - cos (π / r) ^ 2 := by
  simp [trailing3, schlafligram, det_fin_three]
  linarith [sin_sq_add_cos_sq (π / q)]

```

A.17 PlatonicSolids/D3IffPlatonic.lean

$D_3 > 0$ iff the cell is Platonic (58 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/

import PlatonicSolids.Angles
import PlatonicSolids.LeadngMinors

```

```

import PlatonicSolids.Reciprocal
import PlatonicSolids.TrailingMinor

/-!
Paper §3.3.  $D_3 > 0$  iff the cell is Platonic; the trailing  $3 \times 3$  minor
is the same statement for the vertex figure.
-/

open Real

lemma angle_mem_Icc {n : ℕ} (hn : 3 ≤ n) :
   $\pi / n \in \text{Set.Icc } (-(\pi / 2)) (\pi / 2) :=$ 
  ⟨by linarith [angle_pos (Nat.succ_le_of_lt (Nat.zero_lt_of_lt hn)), pi_pos],
    (angle_lt_pi_div_two hn).le⟩

lemma complement_mem_Icc {n : ℕ} (hn : 3 ≤ n) :
   $\pi / 2 - \pi / n \in \text{Set.Icc } (-(\pi / 2)) (\pi / 2) :=$ 
  ⟨by linarith [angle_lt_pi_div_two hn, pi_pos],
    by linarith [angle_pos (Nat.succ_le_of_lt (Nat.zero_lt_of_lt hn))]⟩

/--  $\sin(\pi/p) > \cos(\pi/q)$  iff  $1/p + 1/q > 1/2$ . -/
lemma sin_gt_cos_iff_reciprocal {p q : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) :
   $\sin(\pi / p) > \cos(\pi / q) \leftrightarrow 1 / (p : \mathbb{R}) + 1 / q > 1 / 2 :=$  by
  have hp0 :  $(0 : \mathbb{R}) < p :=$  Nat.cast_pos.mpr (Nat.zero_lt_of_lt hp)
  have hq0 :  $(0 : \mathbb{R}) < q :=$  Nat.cast_pos.mpr (Nat.zero_lt_of_lt hq)
  rw [← sin_pi_div_two_sub ( $\pi / q$ ), gt_iff_lt, gt_iff_lt]
  rw [strictMonoOn_sin.lt_iff_lt (complement_mem_Icc hq) (angle_mem_Icc hp)]
  constructor
  · intro h
    have hsum :  $\pi / p + \pi / q > \pi / 2 :=$  by linarith
    have hmul :  $\pi * (1 / (p : \mathbb{R}) + 1 / q) > \pi * (1 / 2) :=$  by
      convert hsum using 1 <=> field_simp
    exact lt_of_mul_lt_mul_left hmul pi_pos.le
  · intro h
    have hmul :  $\pi * (1 / (p : \mathbb{R}) + 1 / q) > \pi * (1 / 2) :=$ 
      mul_lt_mul_of_pos_left h pi_pos
    have hsum :  $\pi / p + \pi / q > \pi / 2 :=$  by
      convert hmul using 1 <=> field_simp
    linarith

/-- Leading  $3 \times 3$  minor  $D_3 > 0$  iff  $\{p, q\}$  is Platonic. -/
theorem leadingMinor_three_pos_iff {p q r : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) :
   $0 < \text{leadingMinor } 3 \text{ (by decide) } p \ q \ r \leftrightarrow \text{IsPlatonicPair } p \ q :=$  by
  rw [leadingMinor_three, platonic_pair_iff_reciprocal hp hq, sub_pos,
    sq_lt_sq0 (cos_angle_pos hq).le (sin_angle_pos hp).le]
  exact sin_gt_cos_iff_reciprocal hp hq

/-- Trailing  $3 \times 3$  minor  $> 0$  iff the vertex figure  $\{q, r\}$  is Platonic. -/
theorem trailing3_pos_iff {p q r : ℕ} (hq : 3 ≤ q) (hr : 3 ≤ r) :
   $0 < (\text{trailing3 } p \ q \ r).\text{det} \leftrightarrow \text{IsPlatonicPair } q \ r :=$  by
  rw [trailing3_det, ← leadingMinor_three (p := q) (q := r) (r := p)]
  exact leadingMinor_three_pos_iff hq hr

```

A.18 PlatonicSolids/QuadForm.lean

Quadratic form of the Gram matrix (26 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.GramMatrix

```

```

/-!
Paper §3.3. The quadratic form of the Schläfli Gram matrix.
-/

open scoped Matrix
open Real Matrix

private lemma sum_fin4 (f : Fin 4 → ℝ) :  $\sum i, f i = f 0 + f 1 + f 2 + f 3$  := by
  rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_succ]
  simp
  abel

lemma schlafliGram_quad (p q r : ℕ) (x : Fin 4 → ℝ) :
  star x ·v (schlafliGram p q r ·v x) =
    x 0 ^ 2 + x 1 ^ 2 + x 2 ^ 2 + x 3 ^ 2
    - 2 * cos (π / p) * x 0 * x 1
    - 2 * cos (π / q) * x 1 * x 2
    - 2 * cos (π / r) * x 2 * x 3 := by
  simp [schlafliGram, dotProduct, mulVec, sum_fin4]
  ring

```

A.19 PlatonicSolids/CompletingSquare.lean

LDL / completing-the-square identity (55 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Tactic

/-!
Paper §3.3. Completing the square (LDL) for the rank-4 tridiagonal
quadratic form that the Schläfli Gram matrix induces.
-/

/-- Weighted sum of squares after the triangular change of variables. -/
lemma band_quad_squares (α β γ : ℝ) (x0 x1 x2 x3 : ℝ)
  (h2 : 1 - α ^ 2 ≠ 0) (h3 : 1 - α ^ 2 - β ^ 2 ≠ 0) :
  x0 ^ 2 + x1 ^ 2 + x2 ^ 2 + x3 ^ 2 + 2 * α * x0 * x1 + 2 * β * x1 * x2 +
    2 * γ * x2 * x3 =
    (x0 + α * x1) ^ 2
    + (1 - α ^ 2) * (x1 + β / (1 - α ^ 2) * x2) ^ 2
    + ((1 - α ^ 2 - β ^ 2) / (1 - α ^ 2)) *
      (x2 + γ * (1 - α ^ 2) / (1 - α ^ 2 - β ^ 2) * x3) ^ 2
    + ((1 - α ^ 2 - β ^ 2 - γ ^ 2 * (1 - α ^ 2)) / (1 - α ^ 2 - β ^ 2)) *
      x3 ^ 2 := by
  field_simp [h2, h3]
  ring

lemma weighted_squares_pos {a b c d c1 c2 c3 : ℝ}
  (hc1 : 0 < c1) (hc2 : 0 < c2) (hc3 : 0 < c3)
  (h : a ≠ 0 ∨ b ≠ 0 ∨ c ≠ 0 ∨ d ≠ 0) :
  0 < a ^ 2 + c1 * b ^ 2 + c2 * c ^ 2 + c3 * d ^ 2 := by
  rcases h with ha | hb | hc | hd
  · nlinarith [sq_pos_of_ne_zero ha, sq_nonneg b, sq_nonneg c, sq_nonneg d]
  · nlinarith [sq_pos_of_ne_zero hb, sq_nonneg a, sq_nonneg c, sq_nonneg d]
  · nlinarith [sq_pos_of_ne_zero hc, sq_nonneg a, sq_nonneg b, sq_nonneg d]
  · nlinarith [sq_pos_of_ne_zero hd, sq_nonneg a, sq_nonneg b, sq_nonneg c]

/-- The triangular change of variables is invertible. -/
lemma ldl_coords_eq_zero {α β γ x0 x1 x2 x3 : ℝ}
  (_h2 : 1 - α ^ 2 ≠ 0) (_h3 : 1 - α ^ 2 - β ^ 2 ≠ 0)

```

```

(h01 : x0 + α * x1 = 0)
(h12 : x1 + β / (1 - α ^ 2) * x2 = 0)
(h23 : x2 + γ * (1 - α ^ 2) / (1 - α ^ 2 - β ^ 2) * x3 = 0)
(h3z : x3 = 0) :
x0 = 0 ∧ x1 = 0 ∧ x2 = 0 ∧ x3 = 0 := by
have hx3 : x3 = 0 := h3z
have hx2 : x2 = 0 := by
  rw [hx3] at h23
  simp using h23
have hx1 : x1 = 0 := by
  rw [hx2] at h12
  simp at h12
  exact h12
have hx0 : x0 = 0 := by
  rw [hx1] at h01
  simp using h01
exact ⟨hx0, hx1, hx2, hx3⟩

```

A.20 PlatonicSolids/DetFormula.lean

$\det M = \Delta$ (43 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.LinearAlgebra.Matrix.Determinant.Basic
import PlatonicSolids.GramMatrix

/-!
Paper §3.4. The determinant of the rank-4 Schläfli Gram matrix is
`Δ = sin²(π/p) sin²(π/r) - cos²(π/q)`.
-/

open scoped Matrix
open Real Matrix

private lemma sum_fin4 (f : Fin 4 → ℝ) : ∑ i, f i = f 0 + f 1 + f 2 + f 3 := by
  rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_succ]
  simp
  abel

/-- Determinant of the tridiagonal Gram pattern. -/
lemma det_band (a b c : ℝ) :
  (!! [1, a, 0, 0;
    a, 1, b, 0;
    0, b, 1, c;
    0, 0, c, 1] : Matrix (Fin 4) (Fin 4) ℝ).det =
  (1 - a ^ 2) * (1 - c ^ 2) - b ^ 2 := by
  rw [det_succ_row_zero, sum_fin4]
  simp [det_fin_three, Fin.succAbove]
  ring

lemma schlafliGram_det (p q r : ℕ) :
  (schlafliGram p q r).det = schlafliDet p q r := by
  unfold schlafliGram schlafliDet
  rw [det_band]
  have hp : 1 - (-cos (π / p)) ^ 2 = sin (π / p) ^ 2 := by linarith [sin_sq_add_cos_sq (π / p)]
  have hr : 1 - (-cos (π / r)) ^ 2 = sin (π / r) ^ 2 := by linarith [sin_sq_add_cos_sq (π / r)]
  rw [hp, hr]
  ring

lemma leadingMinor_four (p q r : ℕ) :

```

```
(schlafliGram p q r).det = schlafliDet p q r :=
schlafliGram_det p q r
```

A.21 PlatonicSolids/Sylvester.lean

Sylvester: positive definite iff leading minors are positive (100 lines).

```
/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import Mathlib.Analysis.Matrix.PosDef
import PlatonicSolids.CompletingSquare
import PlatonicSolids.DetFormula
import PlatonicSolids.LeadngMinors
import PlatonicSolids.QuadForm

/--
Paper §3.3. Sylvester's criterion for the Schläfli Gram matrix:
positive-definite iff the four leading principal minors are positive.
`PosDef →` minors uses Mathlib (`PosDef.submatrix`, `PosDef.det_pos`);
the converse is the completing-the-square identity.
-/

open scoped Matrix
open Real Matrix

lemma leadingMinors_pos_of_posDef {p q r : ℕ} (h : (schlafliGram p q r).PosDef) :
  0 < leadingMinor 1 (by decide) p q r ∧
  0 < leadingMinor 2 (by decide) p q r ∧
  0 < leadingMinor 3 (by decide) p q r ∧
  0 < (schlafliGram p q r).det := by
  have h1 := h.submatrix (Fin.castLE_injective (by decide : 1 ≤ 4))
  have h2 := h.submatrix (Fin.castLE_injective (by decide : 2 ≤ 4))
  have h3 := h.submatrix (Fin.castLE_injective (by decide : 3 ≤ 4))
  refine ⟨?, ?, ?, PosDef.det_pos h⟩
  · simp [leadingMinor, leadingSubmatrix] using PosDef.det_pos h1
  · simp [leadingMinor, leadingSubmatrix] using PosDef.det_pos h2
  · simp [leadingMinor, leadingSubmatrix] using PosDef.det_pos h3

/-- Converse Sylvester for this Gram family: positive leading minors ⇒ `PosDef`. -/
theorem schlafliGram_posDef_of_minors {p q r : ℕ}
  (h1 : 0 < leadingMinor 1 (by decide) p q r)
  (h2 : 0 < leadingMinor 2 (by decide) p q r)
  (h3 : 0 < leadingMinor 3 (by decide) p q r)
  (h4 : 0 < (schlafliGram p q r).det) :
  (schlafliGram p q r).PosDef := by
  have hD1 : leadingMinor 1 (by decide) p q r = 1 := leadingMinor_one p q r
  have : (0 : ℝ) < 1 := hD1 ▸ h1
  clear this
  refine posDef_iff_dotProduct_mulVec.mpr ⟨schlafliGram_isHermitian p q r, ?_⟩
  intro x hx
  set α := -cos (π / p)
  set β := -cos (π / q)
  set γ := -cos (π / r)
  have hα : 1 - α ^ 2 = sin (π / p) ^ 2 := by
    simp [α]; linarith [sin_sq_add_cos_sq (π / p)]
  have hβ : 1 - α ^ 2 - β ^ 2 = sin (π / p) ^ 2 - cos (π / q) ^ 2 := by
    simp [α, β]; linarith [sin_sq_add_cos_sq (π / p)]
  have hD2 : leadingMinor 2 (by decide) p q r = sin (π / p) ^ 2 := leadingMinor_two p q r
  have hD3 : leadingMinor 3 (by decide) p q r =
    sin (π / p) ^ 2 - cos (π / q) ^ 2 := leadingMinor_three p q r
  have h2ne : 1 - α ^ 2 ≠ 0 := by
```

```

    rw [hα]; exact ne_of_gt (hD2 ▸ h2)
have h3ne : 1 - α ^ 2 - β ^ 2 ≠ 0 := by
    rw [hβ]; exact ne_of_gt (hD3 ▸ h3)
have hc1 : 0 < 1 - α ^ 2 := by rw [hα]; exact hD2 ▸ h2
have hc2 : 0 < 1 - α ^ 2 - β ^ 2 := by rw [hβ]; exact hD3 ▸ h3
have hγ : 1 - γ ^ 2 = sin (π / r) ^ 2 := by
    simp [γ]; linarith [sin_sq_add_cos_sq (π / r)]
have hc3 : 0 < 1 - α ^ 2 - β ^ 2 - γ ^ 2 * (1 - α ^ 2) := by
    have : 1 - α ^ 2 - β ^ 2 - γ ^ 2 * (1 - α ^ 2) =
        (1 - α ^ 2) * (1 - γ ^ 2) - β ^ 2 := by ring
    rw [this, hα, hγ]
have hβsq : β ^ 2 = cos (π / q) ^ 2 := by simp [β]
rw [hβsq]
have hdet : sin (π / p) ^ 2 * sin (π / r) ^ 2 - cos (π / q) ^ 2 =
    (schlafliGram p q r).det := by
    rw [schlafliGram_det, schlafliDet]
rw [hdet]
exact h4
have hQ : star x ·v (schlafliGram p q r *v x) =
    (x 0 + α * x 1) ^ 2
    + (1 - α ^ 2) * (x 1 + β / (1 - α ^ 2) * x 2) ^ 2
    + ((1 - α ^ 2 - β ^ 2) / (1 - α ^ 2)) *
        (x 2 + γ * (1 - α ^ 2) / (1 - α ^ 2 - β ^ 2) * x 3) ^ 2
    + ((1 - α ^ 2 - β ^ 2 - γ ^ 2 * (1 - α ^ 2)) / (1 - α ^ 2 - β ^ 2)) *
        x 3 ^ 2 := by
    calc star x ·v (schlafliGram p q r *v x)
        = x 0 ^ 2 + x 1 ^ 2 + x 2 ^ 2 + x 3 ^ 2
          - 2 * cos (π / p) * x 0 * x 1
          - 2 * cos (π / q) * x 1 * x 2
          - 2 * cos (π / r) * x 2 * x 3 := schlafliGram_quad p q r x
    _ = x 0 ^ 2 + x 1 ^ 2 + x 2 ^ 2 + x 3 ^ 2
        + 2 * α * x 0 * x 1 + 2 * β * x 1 * x 2 + 2 * γ * x 2 * x 3 := by
        simp [α, β, γ]; ring
    _ = _ := band_quad_squares α β γ (x 0) (x 1) (x 2) (x 3) h2ne h3ne
rw [hQ]
refine weighted_squares_pos hc1 (div_pos hc2 hc1) (div_pos hc3 hc2) ?_
by_contra hforms
push Not at hforms
obtain ⟨h01, h12, h23, h3z⟩ := hforms
obtain ⟨hx0, hx1, hx2, hx3⟩ := ldl_coords_eq_zero h2ne h3ne h01 h12 h23 h3z
have : x = 0 := by
    ext i
    fin_cases i <|> simp [hx0, hx1, hx2, hx3]
exact hx this

```

A.22 PlatonicSolids/TrigValues.lean

Exact
 $\sin^2(\frac{\pi}{n})$ and
 $\cos^2(\frac{\pi}{n})$ for n
in
3, 4, 5 (63 lines).

/-

Copyright (c) 2026 Lars Warren Ericson.

Released under Apache 2.0 license as described in the file LICENSE.

-/

import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic

```

/-!
Paper §4. Exact  $\sin^2(\pi/n)$  and  $\cos^2(\pi/n)$  for  $n \in \{3,4,5\}$ , from
Mathlib's sin_pi_div_* / cos_pi_div_*.
-/

open Real

lemma sq_cos_pi_div_three :  $\cos(\pi / 3)^2 = 1 / 4$  := by
  rw [cos_pi_div_three]; norm_num

lemma sq_sin_pi_div_four :  $\sin(\pi / 4)^2 = 1 / 2$  := by
  rw [sin_pi_div_four, div_pow, sq_sqrt (by norm_num : (0 : ℝ) ≤ 2)]; norm_num

lemma sq_cos_pi_div_four :  $\cos(\pi / 4)^2 = 1 / 2$  := by
  rw [cos_pi_div_four, div_pow, sq_sqrt (by norm_num : (0 : ℝ) ≤ 2)]; norm_num

lemma sq_cos_pi_div_five :  $\cos(\pi / 5)^2 = (3 + \sqrt{5}) / 8$  := by
  rw [cos_pi_div_five]
  have :  $(1 + \sqrt{5})^2 = 6 + 2 * \sqrt{5}$  := by
    ring_nf
    rw [sq_sqrt (by norm_num : (0 : ℝ) ≤ 5)]
    ring
  field_simp
  rw [this]
  ring

lemma sq_sin_pi_div_five :  $\sin(\pi / 5)^2 = (5 - \sqrt{5}) / 8$  := by
  have h :  $\sin(\pi / 5)^2 = 1 - \cos(\pi / 5)^2$  := by
    linarith [sin_sq_add_cos_sq (π / 5)]
  rw [h, sq_cos_pi_div_five]
  field_simp
  ring

lemma sqrt5_bounds :  $2 < \sqrt{5} \wedge \sqrt{5} < 3$  := by
  constructor
  · rw [lt_sqrt (by norm_num)]; norm_num
  · rw [sqrt_lt' (by norm_num)]; norm_num

lemma three_mul_sqrt5_lt_seven :  $3 * \sqrt{5} < 7$  := by
  have h :  $(3 * \sqrt{5})^2 < (7 : ℝ)^2$  := by
    calc (3 *  $\sqrt{5}$ )^2 = 9 * ( $\sqrt{5}^2$ ) := by ring
    _ = 9 * 5 := by rw [sq_sqrt (by norm_num)]
    _ = 45 := by norm_num
    _ < 49 := by norm_num
    _ = (7 : ℝ)^2 := by norm_num
  have :  $0 \leq 3 * \sqrt{5}$  := by positivity
  nlinarith

lemma two_mul_sqrt5_gt_three :  $3 < 2 * \sqrt{5}$  := by
  have h :  $(3 : ℝ)^2 < (2 * \sqrt{5})^2$  := by
    calc (3 : ℝ)^2 = 9 := by norm_num
    _ < 20 := by norm_num
    _ = 4 * 5 := by norm_num
    _ = 4 * ( $\sqrt{5}^2$ ) := by rw [sq_sqrt (by norm_num)]
    _ = (2 *  $\sqrt{5}$ )^2 := by ring
  have :  $0 \leq 2 * \sqrt{5}$  := by positivity
  nlinarith

```

A.23 PlatonicSolids/CompatibleTriples.lean

The eleven Platonic-compatible Schläfli triples (64 lines).

```
/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.PlatonicPair

/-!
Paper §4. The eleven Schläfli triples whose cell and vertex figure
are both Platonic.
-/

/-- Compatible middle index: 11 of the 25 Platonic-pair products survive. -/
theorem platonic_compatible_eleven {p q r : ℕ}
  (hc : IsPlatonicPair p q) (hv : IsPlatonicPair q r) :
  p = 3 ∧ q = 3 ∧ r = 3 ∨
  p = 3 ∧ q = 3 ∧ r = 4 ∨
  p = 3 ∧ q = 3 ∧ r = 5 ∨
  p = 3 ∧ q = 4 ∧ r = 3 ∨
  p = 4 ∧ q = 3 ∧ r = 3 ∨
  p = 4 ∧ q = 3 ∧ r = 4 ∨
  p = 4 ∧ q = 3 ∧ r = 5 ∨
  p = 3 ∧ q = 5 ∧ r = 3 ∨
  p = 5 ∧ q = 3 ∧ r = 3 ∨
  p = 5 ∧ q = 3 ∧ r = 4 ∨
  p = 5 ∧ q = 3 ∧ r = 5 := by
  rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
  · -- cell {3,3}
    rcases hv with ⟨_, rfl⟩ | ⟨_, rfl⟩ | ⟨h, _⟩ | ⟨_, rfl⟩ | ⟨h, _⟩
    · exact .inl ⟨rfl, rfl, rfl⟩
    · exact .inr <| .inl ⟨rfl, rfl, rfl⟩
    · cases h
    · exact .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
    · cases h
  · -- cell {3,4}
    rcases hv with ⟨h, _⟩ | ⟨h, _⟩ | ⟨_, rfl⟩ | ⟨h, _⟩ | ⟨h, _⟩
    · cases h
    · cases h
    · exact .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
    · cases h
    · cases h
  · -- cell {4,3}
    rcases hv with ⟨_, rfl⟩ | ⟨_, rfl⟩ | ⟨h, _⟩ | ⟨_, rfl⟩ | ⟨h, _⟩
    · exact .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
    · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
    · cases h
    · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
    · cases h
  · -- cell {3,5}
    rcases hv with ⟨h, _⟩ | ⟨h, _⟩ | ⟨h, _⟩ | ⟨h, _⟩ | ⟨_, rfl⟩
    · cases h
    · cases h
    · cases h
    · cases h
    · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · -- cell {5,3}
    rcases hv with ⟨_, rfl⟩ | ⟨_, rfl⟩ | ⟨h, _⟩ | ⟨_, rfl⟩ | ⟨h, _⟩
    · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inr <|
      .inl ⟨rfl, rfl, rfl⟩
    · exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inr <|
      .inr <| .inl ⟨rfl, rfl, rfl⟩
```

```

· cases h
· exact .inr <| .inr <| .inr <| .inr <| .inr <| .inr <| .inr <|
  .inr <| .inr (rfl, rfl, rfl)
· cases h

```

A.24 PlatonicSolids/SignTable.lean

Sign of Δ on those eleven triples (121 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.GramMatrix
import PlatonicSolids.TrigValues

/-!
Paper §4. Sign of `Δ` on the eleven Platonic-compatible triples.
-/

open Real

private lemma unfold_det (p q r : ℕ) :
  schlafliDet p q r = sin (π / p) ^ 2 * sin (π / r) ^ 2 - cos (π / q) ^ 2 :=
  rfl

private lemma cast_angles :
  sin (π / (3 : ℕ)) ^ 2 = sin (π / 3) ^ 2 ∧
  sin (π / (4 : ℕ)) ^ 2 = sin (π / 4) ^ 2 ∧
  sin (π / (5 : ℕ)) ^ 2 = sin (π / 5) ^ 2 ∧
  cos (π / (3 : ℕ)) ^ 2 = cos (π / 3) ^ 2 ∧
  cos (π / (4 : ℕ)) ^ 2 = cos (π / 4) ^ 2 ∧
  cos (π / (5 : ℕ)) ^ 2 = cos (π / 5) ^ 2 := by
  simp [Nat.cast_ofNat]

lemma schlafliDet_333 : schlafliDet 3 3 3 = 5 / 16 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.2.2.1, sq_sin_pi_div_three, sq_cos_pi_div_three]
  norm_num

lemma schlafliDet_334 : schlafliDet 3 3 4 = 1 / 8 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_three, sq_sin_pi_div_four, sq_cos_pi_div_three]
  norm_num

lemma schlafliDet_433 : schlafliDet 4 3 3 = 1 / 8 := by
  rw [unfold_det, cast_angles.2.1, cast_angles.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_four, sq_sin_pi_div_three, sq_cos_pi_div_three]
  norm_num

lemma schlafliDet_343 : schlafliDet 3 4 3 = 1 / 16 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.2.2.2.1, sq_sin_pi_div_three, sq_cos_pi_div_four]
  norm_num

lemma schlafliDet_335 : schlafliDet 3 3 5 = (7 - 3 * √5) / 32 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.2.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_three, sq_sin_pi_div_five, sq_cos_pi_div_three]
  ring

lemma schlafliDet_533 : schlafliDet 5 3 3 = (7 - 3 * √5) / 32 := by
  rw [unfold_det, cast_angles.2.2.1, cast_angles.1, cast_angles.2.2.2.1,
    sq_sin_pi_div_five, sq_sin_pi_div_three, sq_cos_pi_div_three]
  ring

```

```

lemma schlafliDet_434 : schlafliDet 4 3 4 = 0 := by
  rw [unfold_det, cast_angles.2.1, cast_angles.2.2.2.1, sq_sin_pi_div_four, sq_cos_pi_div_three]
  norm_num

lemma schlafliDet_353 : schlafliDet 3 5 3 = (3 - 2 * √5) / 16 := by
  rw [unfold_det, cast_angles.1, cast_angles.2.2.2.2.2, sq_sin_pi_div_three, sq_cos_pi_div_five]
  ring

lemma schlafliDet_435 : schlafliDet 4 3 5 = (1 - √5) / 16 := by
  rw [unfold_det, cast_angles.2.1, cast_angles.2.2.1, cast_angles.2.2.2.1, sq_sin_pi_div_four, sq_sin_pi_div_five, sq_cos_pi_div_three]
  ring

lemma schlafliDet_534 : schlafliDet 5 3 4 = (1 - √5) / 16 := by
  rw [unfold_det, cast_angles.2.2.1, cast_angles.2.1, cast_angles.2.2.2.1, sq_sin_pi_div_five, sq_sin_pi_div_four, sq_cos_pi_div_three]
  ring

lemma schlafliDet_535 : schlafliDet 5 3 5 = (14 - 10 * √5) / 64 := by
  rw [unfold_det, cast_angles.2.2.1, cast_angles.2.2.2.1, sq_sin_pi_div_five, sq_cos_pi_div_three]
  have : ((5 - √5) / 8) ^ 2 = (30 - 10 * √5) / 64 := by
    field_simp
    ring_nf
    rw [sq_sqrt (by norm_num : (0 : ℝ) ≤ 5)]
    ring
  rw [← pow_two, this]
  ring

lemma schlafliDet_333_pos : 0 < schlafliDet 3 3 3 := by rw [schlafliDet_333]; norm_num
lemma schlafliDet_334_pos : 0 < schlafliDet 3 3 4 := by rw [schlafliDet_334]; norm_num
lemma schlafliDet_433_pos : 0 < schlafliDet 4 3 3 := by rw [schlafliDet_433]; norm_num
lemma schlafliDet_343_pos : 0 < schlafliDet 3 4 3 := by rw [schlafliDet_343]; norm_num

lemma schlafliDet_335_pos : 0 < schlafliDet 3 3 5 := by
  rw [schlafliDet_335]
  have : 0 < 7 - 3 * √5 := by linarith [three_mul_sqrt5_lt_seven]
  positivity

lemma schlafliDet_533_pos : 0 < schlafliDet 5 3 3 := by
  rw [schlafliDet_533]
  have : 0 < 7 - 3 * √5 := by linarith [three_mul_sqrt5_lt_seven]
  positivity

lemma schlafliDet_434_nonpos : schlafliDet 4 3 4 ≤ 0 := by
  rw [schlafliDet_434]

lemma schlafliDet_353_neg : schlafliDet 3 5 3 < 0 := by
  rw [schlafliDet_353]
  have : 3 - 2 * √5 < 0 := by linarith [two_mul_sqrt5_gt_three]
  exact div_neg_of_neg_of_pos this (by positivity)

lemma schlafliDet_435_neg : schlafliDet 4 3 5 < 0 := by
  rw [schlafliDet_435]
  have : 1 - √5 < 0 := by linarith [sqrt5_bounds.1]
  exact div_neg_of_neg_of_pos this (by positivity)

lemma schlafliDet_534_neg : schlafliDet 5 3 4 < 0 := by
  rw [schlafliDet_534]

```

```

have : 1 -  $\sqrt{5}$  < 0 := by linarith [sqrt5_bounds.1]
exact div_neg_of_neg_of_pos this (by positivity)

lemma schlaflDet_535_neg : schlaflDet 5 3 5 < 0 := by
  rw [schlaflDet_535]
  have : 14 - 10 *  $\sqrt{5}$  < 0 := by
    have : 14 < 10 *  $\sqrt{5}$  := by nlinarith [sqrt5_bounds.1]
    linarith
  exact div_neg_of_neg_of_pos this (by positivity)

```

A.25 PlatonicSolids/Regular4.lean

The six regular convex 4-polytopes (28 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.PlatonicPair

/-!
Paper §4, Group I. The six regular convex 4-polytopes.
-/

/-- Schläfli triples of the six regular convex 4-polytopes. -/
def IsRegular4Polytope (p q r : ℕ) : Prop :=
  (p = 3 ∧ q = 3 ∧ r = 3) ∨
  (p = 4 ∧ q = 3 ∧ r = 3) ∨
  (p = 3 ∧ q = 3 ∧ r = 4) ∨
  (p = 3 ∧ q = 4 ∧ r = 3) ∨
  (p = 5 ∧ q = 3 ∧ r = 3) ∨
  (p = 3 ∧ q = 3 ∧ r = 5)

theorem IsRegular4Polytope.ge_three {p q r : ℕ} (h : IsRegular4Polytope p q r) :
  3 ≤ p ∧ 3 ≤ q ∧ 3 ≤ r := by
  rcases h with ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ <;> decide

theorem IsRegular4Polytope.platonic {p q r : ℕ} (h : IsRegular4Polytope p q r) :
  IsPlatonicPair p q ∧ IsPlatonicPair q r := by
  rcases h with ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ <;> simp [IsPlatonicPair]

```

A.26 PlatonicSolids/Classification.lean

$\Delta > 0$ (equivalently M positive definite) iff one of the six (84 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson.
Released under Apache 2.0 license as described in the file LICENSE.
-/
import PlatonicSolids.Angles
import PlatonicSolids.CompatibleTriples
import PlatonicSolids.D3IffPlatonic
import PlatonicSolids.Regular4
import PlatonicSolids.SignTable
import PlatonicSolids.Sylvester
import PlatonicSolids.TrailingMinor

/-!
Paper §4. The six regular convex 4-polytopes are exactly the
Platonic-compatible triples with  $\Delta > 0$ , equivalently those whose

```

```

Schläfli Gram matrix is positive definite.
-/

open Real Matrix

/-- Palomar compared 4D theorem: Platonic cell and vertex figure with
`Δ > 0` force one of the six regular convex 4-polytopes. -/
theorem regular_polychora_classification (p q r : ℕ)
  (h_cell : IsPlatonicPair p q)
  (h_vf : IsPlatonicPair q r)
  (h_det : 0 < schlafliDet p q r) :
  IsRegular4Polytope p q r := by
  rcases platonic_compatible_eleven h_cell h_vf with
  ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  · exact .inl ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inr <| .inr <| .inr <| .inr ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact (not_lt_of_ge schlafliDet_434_nonpos h_det).elim
  · exact (lt_asymm schlafliDet_435_neg h_det).elim
  · exact (lt_asymm schlafliDet_353_neg h_det).elim
  · exact .inr <| .inr <| .inr <| .inr <| .inl ⟨rfl, rfl, rfl⟩
  · exact (lt_asymm schlafliDet_534_neg h_det).elim
  · exact (lt_asymm schlafliDet_535_neg h_det).elim

theorem schlafliDet_pos_of_regular {p q r : ℕ} (h : IsRegular4Polytope p q r) :
  0 < schlafliDet p q r := by
  rcases h with ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩ | ⟨rfl, rfl, rfl⟩
  · exact schlafliDet_333_pos
  · exact schlafliDet_433_pos
  · exact schlafliDet_334_pos
  · exact schlafliDet_343_pos
  · exact schlafliDet_533_pos
  · exact schlafliDet_335_pos

theorem regular_of_posDef {p q r : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) (hr : 3 ≤ r)
  (h : (schlafliGram p q r).PosDef) :
  IsRegular4Polytope p q r := by
  obtain ⟨_, _, h3, h4⟩ := leadingMinors_pos_of_posDef h
  have hcell : IsPlatonicPair p q := (leadingMinor_three_pos_iff hp hq).mpr h3
  have hvf : IsPlatonicPair q r := by
    have hs := h.submatrix (Fin.succ_injective (n := 3))
    have : 0 < (trailing3 p q r).det := by
      simpa [trailing3] using PosDef.det_pos hs
    exact (trailing3_pos_iff hq hr).mpr this
  have hdet : 0 < schlafliDet p q r := by
    rw [← schlafliGram_det]
    exact PosDef.det_pos h
  exact regular_polychora_classification p q r hcell hvf hdet

theorem posDef_of_regular {p q r : ℕ} (h : IsRegular4Polytope p q r) :
  (schlafliGram p q r).PosDef := by
  obtain ⟨hp, hq, _hr⟩ := h.ge_three
  obtain ⟨hcell, _hvf⟩ := h.platonic
  refine schlafliGram_posDef_of_minors ?_ ?_ ?_ ?_
  · rw [leadingMinor_one]; norm_num
  · rw [leadingMinor_two]; exact sq_pos_of_pos (sin_angle_pos hp)
  · exact (leadingMinor_three_pos_iff hp hq).mpr hcell
  · rw [schlafliGram_det]; exact schlafliDet_pos_of_regular h

```

```

/-- Finite regular convex 4-polytopes are exactly the triples `{p,q,r}`
with `{p,q,r} ≥ 3` whose Schläfli Gram matrix is positive definite. -/
theorem regular_polychora_iff {p q r : ℕ} (hp : 3 ≤ p) (hq : 3 ≤ q) (hr : 3 ≤ r) :
  (schlafligram p q r).PosDef ↔ IsRegular4Polytope p q r :=
  ⟨regular_of_posDef hp hq hr, posDef_of_regular⟩

```

A.27 Challenge.lean

Palomar statement of record (deliberate sorries) (113 lines).

```

/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/
import Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic
import Mathlib.LinearAlgebra.Matrix.Notation
import Mathlib.Tactic

/-!
# Palomar statement of record (Platonic solids and regular polychora)

This module states the compared Palomar family and the Mathlib type
surface it depends on. Challenge may use `sorry`; Solution supplies the
proofs.

The selected theorems are:

1. `platonic_solids_3d` - Euler  $(V-E+F=2)$  plus regularity incidence
 $(pF=2E)$ ,  $(qV=2E)$  force a Platonic pair  $((p,q))$ .
2. `edges_pos_of_regular` - the same identities algebraically force
 $(E>0)$ ; non-degeneracy is a theorem, not an axiom.
3. `regular_polychora_classification` - Platonic cells and vertex figures
with positive Schläfli Gram determinant are exactly the six regular
convex 4-polytopes.

## How to read this file

The definitions below are the vocabulary of the claim. A reader who wants
to check what has been proved should read this file and need not read
anything else. `Solution.lean` imports the sorry-free development in
`PlatonicSolids.lean`. Each supporting lemma lives in a section-sized
module under `PlatonicSolids/`, meant to be read next to the matching
paragraph of `PlatonicSolids.md`.
-/

open scoped Matrix
open Real Matrix

/-- The 5 Platonic pairs (p, q):
(3, 3): Tetrahedron
(3, 4): Octahedron
(4, 3): Cube
(3, 5): Icosahedron
(5, 3): Dodecahedron -/
def IsPlatonicPair (p q : ℕ) : Prop :=
  (p = 3 ∧ q = 3) ∨
  (p = 3 ∧ q = 4) ∨
  (p = 4 ∧ q = 3) ∨
  (p = 3 ∧ q = 5) ∨
  (p = 5 ∧ q = 3)

```

```

/-- Canonical bilinear form of the rank-4 Schläfli Coxeter system {p, q, r}. -/
noncomputable def schlafligram (p q r : ℕ) : Matrix (Fin 4) (Fin 4) ℝ :=
  !![1, -cos (π / p), 0, 0;
    -cos (π / p), 1, -cos (π / q), 0;
    0, -cos (π / q), 1, -cos (π / r);
    0, 0, -cos (π / r), 1]

/-- Δ(p, q, r) = sin²(π/p) sin²(π/r) - cos²(π/q). -/
noncomputable def schlaflidet (p q r : ℕ) : ℝ :=
  sin (π / p) ^ 2 * sin (π / r) ^ 2 - cos (π / q) ^ 2

/-- The 6 regular convex 4-polytopes (Schläfli symbols {p, q, r}):
  {3, 3, 3}: 5-cell (4D Simplex)
  {4, 3, 3}: 8-cell (Tesseract / Hypercube)
  {3, 3, 4}: 16-cell (Cross-polytope)
  {3, 4, 3}: 24-cell (Exceptional 4D regular polytope)
  {5, 3, 3}: 120-cell
  {3, 3, 5}: 600-cell -/
def IsRegular4Polytope (p q r : ℕ) : Prop :=
  (p = 3 ∧ q = 3 ∧ r = 3) ∨
  (p = 4 ∧ q = 3 ∧ r = 3) ∨
  (p = 3 ∧ q = 3 ∧ r = 4) ∨
  (p = 3 ∧ q = 4 ∧ r = 3) ∨
  (p = 5 ∧ q = 3 ∧ r = 3) ∨
  (p = 3 ∧ q = 3 ∧ r = 5)

/-- Theorem: A Platonic configuration cannot have zero edges.
  The topological Euler characteristic (χ = 2) and regularity conditions (p, q ≥ 3)
  algebraically force E ≥ 1 without requiring any geometric non-degeneracy axiom. -/
theorem edges_pos_of_regular
  (V E F p q : ℕ)
  (hp : 3 ≤ p) (hq : 3 ≤ q)
  (h_edges_faces : p * F = 2 * E)
  (h_edges_verts : q * V = 2 * E)
  (h_euler : V + F = E + 2) :
  0 < E := by
  sorry

/-- The 3D Main Theorem:
  Any polyhedron satisfying Euler's formula (V - E + F = 2)
  and regularity incidence relations (pF = 2E, qV = 2E)
  must have (p, q) belonging to the 5 Platonic solids. -/
theorem platonic_solids_3d
  (V E F p q : ℕ)
  (hp : 3 ≤ p) (hq : 3 ≤ q)
  (h_edges_faces : p * F = 2 * E)
  (h_edges_verts : q * V = 2 * E)
  (h_euler : V + F = E + 2) :
  IsPlatonicPair p q := by
  sorry

/-- The Main Classification Theorem for 4D Regular Polychora:
  Any triple {p, q, r} whose 3D cells {p, q} and vertex figures {q, r} are
  Platonic solids, and whose Coxeter Gram matrix is positive-definite (Δ > 0),
  is strictly one of the 6 regular convex 4-polytopes. -/
theorem regular_polychora_classification (p q r : ℕ)
  (h_cell : IsPlatonicPair p q)
  (h_vf : IsPlatonicPair q r)
  (h_det : 0 < schlaflidet p q r) :
  IsRegular4Polytope p q r := by
  sorry

```

A.28 Solution.lean

Palomar solution module: imports PlatonicSolids (20 lines).

```
/-
Copyright (c) 2026 Lars Warren Ericson. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Lars Warren Ericson.
-/
import PlatonicSolids

/-!
# Solutions to the Challenge

The declarations of `Challenge.lean`, proved. Importing `PlatonicSolids`
supplies `platonic_solids_3d`, `edges_pos_of_regular`,
`regular_polychora_classification` and the definitions they depend on
(`IsPlatonicPair`, `IsRegular4Polytope`, `schlafliGram`, `schlafliDet`),
with the same names and types as in the Challenge module.
Comparator compares those declarations.

The proofs are split across section-sized modules under `PlatonicSolids/`.
There is no `sorry` in those files.
-/
```